Глава 7

Связывание

7.1. Драйверы компиляторов.
7.2. Статическое связывание.
7.3. Объектные файлы.
7.4. Перемещаемые объектные файлы.
7.5. Идентификаторы и таблицы имен.
7.6. Разрешение ссылок.
7.7. Перемещение.
7.8. Выполняемые объектные файлы.
7.9. Загрузка выполняемых объектных файлов.
7.10. Динамическое связывание с разделяемыми библиотеками.
7.11. Загрузка и связывание с разделяемыми библиотеками из приложений.
7.12. Перемещаемый программный код.
7.13. Подмена библиотечных функций.
7.14. Инструменты для управления объектными файлами.
7.15. Итоги.
Библиографические заметки.
Домашние задания.
Решения упражнений.

Связывание (*компоновка*) – это процесс сборки и объединения различных частей программного кода и данных в единый файл, который можно загрузить в память и запустить на выполнение. Связывание может выполняться *во время компиляции* – после трансляции исходного кода в машинный; *во время загрузки* – сразу после загрузки программы в память *загрузчиком*; и даже *во время выполнения*. В ранних компьютерных системах связывание выполнялось вручную. В современных системах это делается автоматически – специальными программами, которые называются *редакторами связей*, или *компоновщиками*.

Компоновщики играют ключевую роль в разработке программного обеспечения, потому что обеспечивают возможность *раздельной компиляции*. Они избавляют от необходимости создавать большие приложения в одном монолитном файле с исходным кодом и позволяют расчленить его на множество меньших по размеру модулей, которые могут изменяться и компилироваться раздельно. После изменения любых модулей их можно скомпилировать и снова скомпоновать приложение без повторной компиляции остальных файлов.

Ошибки во время компоновки случаются редко, и у студентов не возникает серьезных проблем при разработке маленьких программ во вводных курсах по программированию, поэтому давайте определим, что дает знание и понимание особенностей связывания программ:

- *понимание особенностей работы компоновщиков помогает при создании больших программ*. Программисты, работающие над большими программами, часто сталкиваются с ошибками времени компоновки, вызванными отсутствием модулей, библиотек или несовместимостью версий этих модулей и библиотек. Если не знать, как редактор связей разрешает ссылки на библиотеки, то такие ошибки будут вас ставить в тупик;
- *понимание особенностей работы компоновщиков помогает избежать опасных ошибок программирования*. Решения, которые принимают редакторы связей в Linux при разрешении ссылок, могут незаметно для вас воздействовать на корректность программ. Программы, неправильно определяющие различные глобальные переменные, по умолчанию не вызывают предупреждений компоновщика, но получившийся в результате выполняемый код может производить необъяснимые действия, и такие ошибки чрезвычайно трудно отлаживать. Мы покажем вам, как это происходит и как этого можно избежать;
- *понимание особенностей работы компоновщиков помогает усвоить правила областей видимости в языке*, например понять разницу между глобальными и локальными переменными, смысл объявления переменных или функций статическими;
- *понимание особенностей работы компоновщиков помогает освоить другие важные понятия*. Выполняемые объектные файлы, сформированные редакторами связей, играют важнейшую роль в работе таких системных функций, как загрузка и запуск программ, управление виртуальной памятью, страничная организация памяти и отображение памяти;
- *понимание особенностей работы компоновщиков* дает возможность пользоваться разделяемыми библиотеками. В течение многих лет компоновка считалась простой и неинтересной операцией. Однако вместе с распространением разделяемых библиотек и динамического связывания в современных операционных системах компоновка превратилась в сложный творческий процесс, предоставляющий опытным программистам богатые возможности. Например, многие программные продукты используют разделяемые библиотеки для расширения возможностей двоичных прикладных программ во время выполнения. Кроме того, многие веб-серверы широко используют разделяемые библиотеки для обслуживания динамически меняющегося содержимого.

В этой главе мы подробно рассмотрим все аспекты компоновки, от традиционного статического до динамического связывания с разделяемыми библиотеками во время загрузки и выполнения. Мы опишем работу стандартных механизмов на реальных примерах и покажем, в каких ситуациях проблемы компоновки могут влиять на корректную работу программ. Чтобы объяснения оставались конкретными и понятными, мы будем формулировать наши рассуждения в контексте системы Linux для архитектуры x86-64 с использованием стандартного формата объектных файлов ELF-64 (далее мы будем называть его просто ELF). Однако важно понимать, что основные идеи компоновки универсальны и не зависят от операционной системы, архитектуры набора команд (ISA) или формата представления объектных файлов. Детали могут меняться, но идеи остаются неизменными.

7.1. Драйверы компиляторов

Рассмотрим программу в листинге 7.1. Она будет служить нам простым практическим примером на протяжении всей главы, на котором мы будем демонстрировать особенности работы компоновщиков.

Листинг 7.1. Пример программы 1. Программа состоит из двух файлов с исходным кодом: `main.c` и `sum.c`. Функция `main` инициализирует массив целых чисел, а затем вызывает функцию `sum`, чтобы подсчитать сумму элементов массива

(a) main.c

code/link/main.c

```
1 int sum(int *a, int n);
2
3 int array[2] = {1, 2};
4
5 int main()
6 {
7     int val = sum(array, 2);
8     return val;
9 }
```

code/link/main.c

(b) sum.c

code/link/sum.c

```
1 int sum(int *a, int n)
2 {
3     int i, s = 0;
4
5     for (i = 0; i < n; i++) {
6         s += a[i];
7     }
8     return s;
9 }
```

code/link/sum.c

Большинство систем компиляции предоставляют *драйвер компилятора*, который вызывает препроцессор, компилятор, ассемблер и компоновщик по мере необходимости. Например, собрать программу из примера выше с помощью системы компиляции GNU можно, вызвав драйвер GCC следующей командой:

```
linux> gcc -Og -o prog main.c sum.c
```

На рис. 7.1 показаны действия, выполняемые драйвером по мере преобразования исходного кода программы в выполняемый объектный файл. (Чтобы увидеть выполнение этих шагов во всех деталях, запустите `gcc` с параметром `-v`.) Драйвер сначала запускает препроцессор языка C (`cpp`)[1], который преобразует файл `main.c` с исходным кодом в файл с промежуточным кодом `main.i`:

```
cpp [другие аргументы] main.c /tmp/main.i
```

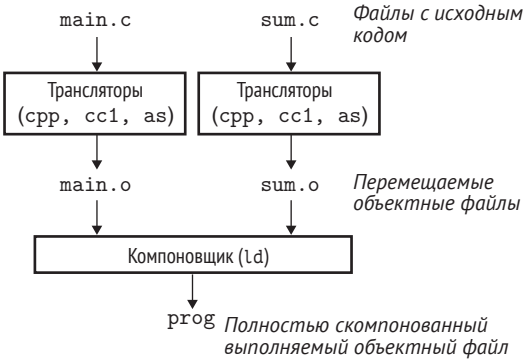


Рис. 7.1. Статическое связывание. Компоновщик объединяет перемещаемые объектные файлы и формирует из них выполняемый объектный файл `prog`

[1] В некоторых версиях GCC препроцессор встроен непосредственно в драйвер.

Далее драйвер вызывает компилятор языка C (cc1), который транслирует промежуточный исходный код из файла main.i в код на языке ассемблера и сохраняет его в файле main.s:

```
cc1 /tmp/main.i -Og [другие аргументы] -o /tmp/main.s
```

Затем драйвер вызывает ассемблер (as), который транслирует код на языке ассемблера из файла main.s в двоичный *перемещаемый объектный код* и сохраняет его в файле main.o:

```
as [другие аргументы] -o /tmp/main.o /tmp/main.s
```

Тот же процесс повторяется для файла sum.c, в результате чего получается файл sum.o. Наконец, драйвер вызывает компоновщика ld, который объединяет main.o и sum.o с другими обязательными системными объектными файлами и создает двоичный выполняемый объектный файл prog:

```
ld -o prog [системные объектные файлы и аргументы] /tmp/main.o /tmp/sum.o
```

Чтобы запустить получившуюся программу, нужно ввести ее имя в командной оболочке Linux:

```
linux> ./prog
```

Командная оболочка обратится к *загрузчику* в операционной системе, который скопирует код и данные из выполняемого файла prog в память и затем передаст управление в начальную точку программы.

7.2. Статическое связывание

Статический компоновщик, такой как программа LD, принимает параметры и группу файлов с перемещаемым объектным кодом и генерирует полностью связанный выполняемый файл, который можно загрузить и запустить. Файлы с перемещаемым объектным кодом содержат различные секции программного кода и данных, каждая из которых представляет собой непрерывную последовательность байтов. Инструкции находятся в одной секции, инициализированные глобальные переменные – в другой, а неинициализированные переменные – в третьей.

Сборка выполняемого файла компоновщиком выполняется в два этапа.

Этап 1

Разрешение ссылок. В объектных файлах содержатся определения *имен* и ссылки на них, где каждое имя соответствует функции, глобальной переменной или *статической переменной* (т. е. любой переменной, объявленной с атрибутом static). Цель разрешения ссылок – связать каждую *ссылку* с соответствующим *определением*.

Этап 2

Перемещение. Компиляторы и ассемблеры генерируют секции программного кода и данных, начиная с нулевого адреса. Компоновщик *перемещает* эти секции, связывая каждое определение имени с адресом в памяти и изменяя все ссылки на эти имена так, чтобы они указывали на соответствующий адрес в памяти. Компоновщик выполняет эти перемещения, слепо следуя подробным инструкциям, сгенерированным ассемблером, которые называются *записями перемещения*.

В последующих разделах эти этапы описываются более подробно. Когда вы перейдете к ним, то имейте в виду некоторые основные факты, связанные с компоновщика-

ми: объектные файлы – это просто совокупности блоков байтов. Одни блоки содержат программный код, другие – данные, а третьи – структуры данных, которые управляют компоновщиком и загрузчиком. Компоновщик объединяет (связывает) блоки, определяет для этих блоков адреса и модифицирует различные ссылки внутри блоков данных и программного кода. Компоновщики имеют минимальное представление о целевой машине, потому что вся основная работа уже проделана компиляторами и ассемблерами, сгенерировавшими объектные файлы.

7.3. Объектные файлы

Объектные файлы могут иметь одну из трех форм:

- *файлы с перемещаемым объектным кодом*. Содержат двоичный код и данные в форме, позволяющей объединить их с другими файлами с перемещаемым объектным кодом, чтобы получить выполняемый объектный код;
- *файлы с выполняемым объектным кодом*. Содержат двоичный код и данные, которые можно загрузить непосредственно в память и запустить на выполнение;
- *файлы с разделяемым объектным кодом*. Особый вид перемещаемого объектного кода, который можно загрузить в память и динамически связать с другими модулями во время загрузки или во время выполнения.

Компиляторы и ассемблеры генерируют перемещаемый объектный код (включая разделяемый объектный код). Компоновщики генерируют выполняемый объектный код. Технически *объектный код* – это последовательность байтов, а *объектный файл* – это объектный код, хранимый в файле на диске. Однако мы будем использовать эти термины как взаимозаменяемые.

В разных системах используются разные *форматы объектных файлов*. В первых версиях Unix, созданных в Bell Labs, использовался формат `a.out`. (И по сей день выполняемые файлы называют `a.out`.) В Windows используется формат переносимых выполняемых файлов (Portable Executable, PE). В Mac OS-X используется формат Mach-O. Современные системы Linux и Unix для архитектуры x86-64 используют *формат выполняемых и компонуемых модулей* (Executable and Linkable Format, ELF). Далее мы сосредоточимся исключительно на формате ELF, однако базовые принципы не зависят от конкретного формата.

7.4. Перемещаемые объектные файлы

На рис. 7.2 показан формат типичного перемещаемого объектного файла ELF. *Заголовок ELF* начинается с 16-байтной последовательности, описывающей размер слова и порядок следования байтов в системе, где был сгенерирован этот файл. Остальная часть заголовка ELF содержит информацию, которая позволяет компоновщику анализировать и интерпретировать объектный файл, в том числе размер заголовка ELF, тип объектного файла (перемещаемый, выполняемый или разделяемый), вид аппаратной архитектуры (например, x86-64), смещение таблицы заголовков секций в файле, размер и количество записей в таблице заголовков секций. Адреса и размеры различных секций описаны в *таблице заголовков секций*, которая содержит элементы фиксированного размера с описанием каждой секции в объектном файле.

Между заголовком ELF и таблицей заголовков секций находятся сами секции. Типичный перемещаемый объектный файл ELF содержит следующие секции:

`.text` – машинный код скомпилированной программы;

`.rodata` – данные, доступные только для чтения, такие как строки формата в вызовах `printf` и таблицах переходов инструкций `switch`;

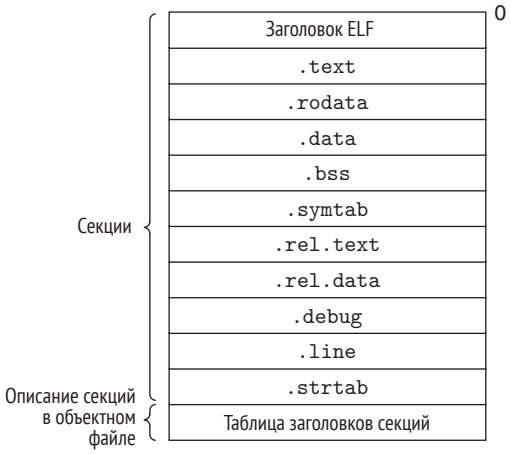


Рис. 7.2. Типичный перемещаемый объектный файл ELF

`.data` – *инициализированные* глобальные переменные. Локальные переменные размещаются в стеке и поэтому не перечисляются ни в секции `.data`, ни в секции `.bss`;

`.bss` – *неинициализированные* глобальные переменные. Фактически эта секция не занимает места в объектном файле, она просто служит «заполнителем места». Форматы объектных файлов различают инициализированные и неинициализированные переменные для экономии памяти: неинициализированные переменные не занимают места в объектном файле на диске. Во время выполнения для них выделяется память, которая предварительно заполняется нулевыми байтами;

`.symtab` – *таблица имен* (symbol table) с информацией о функциях и глобальных переменных в программе, на которые имеются ссылки. Некоторые программисты ошибочно считают, что для создания таблицы имен программа должна компилироваться с параметром `-g`. Но на самом деле каждый перемещаемый объектный файл имеет таблицу имен в секции `.symtab` (если, конечно, программист явно не удалил ее командой STRIP). Однако, в отличие от таблицы идентификаторов в компиляторе, таблица имен `.symtab` не содержит информации о локальных переменных;

`.rel.text` – список адресов в секции `.text`, которые должны быть изменены в процессе компоновки этого объектного файла с другими. В общем случае любая инструкция, вызывающая внешнюю функцию или ссылающаяся на глобальную переменную, должна быть модифицирована. С другой стороны, инструкции, вызывающие локальные функции, не должны модифицироваться. Имейте в виду, что информация о перемещении не требуется в выполняемых объектных файлах и обычно опускается, если пользователь явно не сообщит компоновщику включить ее;

`.rel.data` – информация о перемещении для каждой глобальной переменной, на которую имеется ссылка или которая определяется в данном модуле. В общем случае каждая инициализированная глобальная переменная, начальное значение которой является адресом глобальной переменной или внешней функции, должна быть модифицирована;

`.debug` – таблица имен для отладки с информацией о локальных переменных и определениях типов в программе, глобальных переменных, на которые имеют-

ся ссылки или которые определяются в программе, а также о файле с исходным кодом. Эта секция добавляется в объектный файл, только если драйвер компилятора был вызван с параметром `-g`;

`.line` – таблица соответствий между номерами строк в исходном коде и машинными инструкциями в секции `.text`. Эта секция добавляется в объектный файл, только если драйвер компилятора был вызван с параметром `-g`;

`.strtab` – таблица строк для таблиц имен в секциях `.symtab` и `.debug` и для секции имен в заголовках секций. Таблица строк представляет собой последовательность символьных строк с нулевым символом в конце.

Почему секция с неинициализированными данными называется .bss?

Имя `.bss` для секции с неинициализированными данными широко распространено. Первоначально (приблизительно в 1957 году) это была директива ассемблера IBM 704 «block started by symbol» (начало блока памяти), и это обозначение так и сохранилось. Для простоты запоминания различий между секциями `.data` и `.bss` представляйте «bss» как сокращение от «Better Save Space!» (экономь память).

7.5. Идентификаторы и таблицы имен

Каждый перемещаемый объектный модуль *m* имеет таблицу имен с информацией обо всех именах, которые определяются или на которые имеются ссылки в *m*. С точки зрения компоновщика имеются три вида имен:

- *глобальные имена* (global symbol), которые определены в модуле *m* и на которые могут ссылаться другие модули. Глобальные имена соответствуют *нестатическим* функциям и глобальным переменным;
- глобальные имена, на которые ссылается модуль *m*, но определяемые в других модулях. Такие имена называются *внешними* (external) и соответствуют функциям и переменным, объявленным в других модулях;
- *локальные имена* (local symbol), которые определены и используются исключительно внутри модуля *m*. Эти имена соответствуют статическим функциям и глобальным переменным, объявленным с атрибутом `static`. Они доступны из любой точки внутри модуля *m*, но недоступны из других.

Важно понимать, что локальные имена – это не локальные переменные в программе. Таблица имен в `.symtab` не содержит имен локальных нестатических переменных программы. Последние размещаются в стеке во время выполнения и не представляют интереса для компоновщика.

Интересно отметить, что локальные переменные в функциях, объявленные с атрибутом `static`, размещаются не в стеке. Для каждой такой переменной компилятор выделяет место в `.data` или `.bss` и создает в таблице имен уникальное локальное имя для компоновщика. Например, представьте, что в одном и том же модуле есть пара функций, определяющих статическую локальную переменную `x`:

```
1 int f()
2 {
3     static int x = 0;
4     return x;
5 }
6
```

```
 7 int g()
 8 {
 9     static int x = 1;
10     return x;
11 }
```

В этом случае компилятор создаст пару разных локальных имен, которые подставит в ассемблерный код. Например, для локальной статической переменной в функции `f` он может создать имя `x.1`, и для локальной статической переменной в функции `g` – имя `x.2`.

В таблицах имен, созданных ассемблерами, используются имена, переданные компилятором в ассемблерный файл `.s`. Таблица имен ELF находится в секции `.symtab`. Она содержит массив записей. В листинге 7.2 показан формат каждой такой записи.

Листинг 7.2. Структура записей в таблице имен ELF. Поля `type` и `binding` имеют размеры по 4 бита каждый

code/link/elfstructs.c

```
1 typedef struct {
2     int name;       /* Смещение в таблице строк */
3     char type:4,    /* Функция или данные (4 бита) */
4          binding:4; /* Локальное или глобальное имя (4 бита) */
5     char reserved;  /* Не используется */
6     short section;  /* Индекс заголовка секции */
7     long value;     /* Смещение в секции или абсолютный адрес */
8     long size;      /* Размер объекта в байтах */
9 } Elf64_Symbol;
```

code/link/elfstructs.c

Поле `name` хранит смещение (в байтах) в таблице строк, где находится строка с именем переменной, `value` хранит адрес переменной. Для перемещаемых модулей поле `value` определяет смещение от начала секции, где определен данный объект. Для выполняемых объектных файлов `value` – это абсолютный адрес времени выполнения. Поле `size` хранит размер (в байтах) объекта, `type` определяет тип объекта: `data` или `function`. Таблица имен может также содержать элементы, описывающие отдельные секции и путь к файлу с исходным кодом программы. Поле `binding` определяет вид имени – глобальное или локальное.

Сокрытие имен переменных и функций

Чтобы скрыть объявления переменных и функций в модулях, программисты на C используют атрибут `static`, подобно тому, как в Java и C++ использовали бы объявления `public` и `private`. В языке C роль модулей играют файлы с исходным кодом. Каждая глобальная переменная или функция, объявленная с атрибутом `static`, становится приватной для этого модуля. Точно так же каждая глобальная переменная или функция, объявленная без атрибута `static`, становится общедоступной и может использоваться другими модулями. Хорошей практикой считается защищать переменные и функции с помощью атрибута `static` везде, где это возможно.

Каждое имя связано с некоторой секцией в объектном файле, индекс которой в таблице заголовков секций указывается в поле `section`. Имеются три специальные секции, не имеющие соответствующих записей в таблице заголовков секций: ABS – с неперемещаемыми именами; UNDEF – с неопределенными именами (на которые имеются ссылки в данном объектном модуле, но определяемые в другом модуле); COMMON – с неи-

нициализированными и пока неразмещенными в памяти объектами данных. Для имен в секции COMMON поле `value` задает требования к выравниванию, а `size` определяет минимальный размер. Обратите внимание, что эти специальные секции присутствуют только в перемещаемых объектных файлах – их нет в выполняемых объектных файлах.

Секции COMMON и `.bss` имеют тонкое различие. Современные версии GCC помещают имена в секции COMMON и `.bss` в перемещаемых объектных файлах, используя следующее соглашение:

COMMON	Неинициализированные глобальные переменные
`.bss`	Неинициализированные статические переменные, и глобальные или статические переменные, инициализированные нулевыми значениями

Причина такого, казалось бы, непонятного разделения обусловлена способом разрешения ссылок в компоновщике, о котором рассказывается в разделе 7.6.

Программа GNU READELF – удобный инструмент для просмотра содержимого объектных файлов. Например, вот последние три записи в таблице имен из перемещаемого объектного файла `main.o`, полученного при компиляции программы из листинга 7.1. Первые восемь записей, которые здесь не показаны, – это локальные имена, которые компоновщик использует для внутренних целей.

```
Num:    Value          Size Type    Bind   Vis      Ndx Name
  8: 0000000000000000    24 FUNC    GLOBAL DEFAULT    1 main
  9: 0000000000000000     8 OBJECT  GLOBAL DEFAULT    3 array
 10: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND sum
```

В этом примере можно видеть запись, соответствующую определению глобального имени `main`, 24-байтной функции, находящейся в секции `.text` с нулевым смещением (поле `value`). За ней следует запись, соответствующая определению глобального имени `array`, 4-байтного объекта, находящегося в секции `.data` с нулевым смещением. И последняя запись соответствует ссылке на внешнее имя `sum`. Программа READELF идентифицирует секции по целочисленным индексам. `Ndx=1` соответствует секции `.text`, а `Ndx=3` – секции `.data`.

Листинг 7.3. Пример программы для упражнения 7.1

(a) m.c

code/link/m.c

```
1 void swap();
2
3 int buf[2] = {1, 2};
4
5 int main()
6 {
7     swap();
8     return 0;
9 }
```

code/link/m.c

(b) swap.c

code/link/swap.c

```
 1 extern int buf[];
 2
 3 int *bufp0 = &buf[0];
 4 int *bufp1;
 5
 6 void swap()
 7 {
 8     int temp;
 9
10     bufp1 = &buf[1];
11     temp = *bufp0;
12     *bufp0 = *bufp1;
13     *bufp1 = temp;
14 }
```

code/link/swap.c

Имя	Имеется запись в .symtab?	Тип имени	Модуль, где определено	Секция
buf	_____	_____	_____	_____
bufp0	_____	_____	_____	_____
bufp1	_____	_____	_____	_____
swap	_____	_____	_____	_____
temp	_____	_____	_____	_____

Упражнение 7.1 (решение в конце главы)

В этом упражнении используются модули `m.o` и `swap.o` из листинга 7.3. Для каждого имени, которое определяется или на которое имеется ссылка в `swap.o`, укажите, будет ли для него создана запись в таблице имен в секции `.symtab` в модуле `swap.o`. Если да, то укажите модуль, в котором определяется имя (`swap.o` или `m.o`), тип имени (локальное, глобальное или внешнее) и секцию (`.text`, `.data`, `.bss` или `COMMON`) в этом модуле, где оно находится.

7.6. Разрешение ссылок

Компоновщик разрешает ссылки на имена, связывая каждую ссылку с конкретным определением имени в таблицах имен перемещаемых объектных файлов. Разрешение ссылок на локальные имена, которые определены в том же модуле, где находится сама ссылка, выполняется просто. Компилятор требует, чтобы каждое имя было определено в модуле только один раз. Также компилятор гарантирует уникальность имен локальных статических переменных, которые получит компоновщик.

Разрешение ссылок на глобальные имена – однако, более сложная задача. Встретив имя (переменной или функции), которое не определено в текущем модуле, компилятор предполагает, что это имя определено в некотором другом модуле, генерирует запись в таблицы имен и оставляет его для дальнейшей обработки компоновщиком. Если ни в одном из указанных входных модулей компоновщик не найдет определение имени, на которое имеется ссылка, он выведет сообщение об ошибке (часто загадочное) и завершится. Например, попробуем скомпилировать и скомпоновать в Linux следующий исходный код:

```
1 void foo(void);
2
3 int main() {
4     foo();
5     return 0;
6 }
```

Компилятор успешно выполнит свою часть работы, но компоновщик сообщит об ошибке, не сумев разрешить ссылку на `foo`:

```
linux> gcc -Wall -Og -o linkerror linkerror.c
/tmp/ccSz5uti.o: In function 'main':
/tmp/ccSz5uti.o(.text+0x7): undefined reference to 'foo'
```

Разрешение ссылок на глобальные имена осложняется тем, что одно и то же имя может быть определено в нескольких объектных модулях. В таком случае компоновщик

должен сообщить об ошибке или выбрать одно из определений и отвергнуть остальные. В Linux использован подход, основанный на тесном взаимодействии компилятора, ассемблера и компоновщика, вследствие чего при безалаберном отношении к программированию могут возникать «необъяснимые» ошибки.

Преобразование имен в языках C++ и Java специально для компоновщика

Оба языка, C++ и Java, поддерживают перегрузку методов, то есть позволяют определять методы с одинаковыми именами, но с разными списками параметров. Как компоновщик отличает такие перегруженные функции? Дело в том, что механизм поддержки перегруженных функций в компиляторах языков C++ и Java преобразует имена перегруженных методов с учетом комбинации его параметров и создает уникальные имена специально для компоновщика.

Интересно отметить, что языки C++ и Java используют совместимые схемы преобразования имен. Преобразованное имя класса состоит из количества символов в имени, за которым следует оригинальное имя. Например, имя класса `Foo` будет преобразовано в `3Foo`. Преобразованное имя метода включает оригинальное имя, за которым следует два символа подчеркивания (`__`), затем преобразованное имя класса и далее однобуквенные коды, представляющие типы параметров. Например, имя `Foo::bar(int, long)` будет преобразовано в `bar__3Fooil`. Похожие схемы используются для преобразования имен глобальных переменных и шаблонов.

7.6.1. Как компоновщик разрешает ссылки на повторяющиеся имена

На входе компоновщик получает набор перемещаемых объектных модулей. Каждый из этих модулей определяет набор имен – локальных (видимых только внутри модуля, где они определены) и глобальных (видимых другим модулям). Что случится, если в нескольких модулях определить одинаковые глобальные имена? Далее описывается решение, реализованное в системах компиляции Linux.

Во время компиляции компилятор передает ассемблеру каждое глобальное имя как *строго* или как *слабо* определенное, а ассемблер неявно кодирует эту информацию в таблице имен перемещаемого объектного файла. Имена функций и инициализированных глобальных переменных считаются строго определенными, а неинициализированные глобальные переменные – слабо определенными.

Основываясь на этих понятиях строго и слабо определенных имен, компоновщики в Linux используют следующие правила для обработки повторяющихся имен:

1. Наличие нескольких строго определенных имен недопустимо.
2. При наличии одного строго определенного и нескольких слабо определенных имен выбирается строго определенное имя.
3. При наличии нескольких слабо определенных имен выбирается любое из слабо определенных имен.

Предположим, что нам нужно скомпилировать и скомпоновать следующие два модуля:

```
1 /* foo1.c */
2 int main()
3 {
4     return 0;
5 }
```

```
1 /* bar1.c */
2 int main()
3 {
4     return 0;
5 }
```

В данном случае компоновщик сгенерирует сообщение об ошибке, потому что строго определенное имя `main` встречается несколько раз (правило 1):

```
linux> gcc foo1.c bar1.c
/tmp/ccq2Uxnd.o: In function 'main':
bar1.c:(.text+0x0): multiple definition of 'main'
```

Точно так же компоновщик сгенерирует сообщение об ошибке для следующих модулей, потому что строго определенное имя `x` определено дважды (правило 1):

```
1 /* foo2.c */
2 int x = 15213;
3
4 int main()
5 {
6     return 0;
7 }
```

```
1 /* bar2.c */
2 int x = 15213;
3
4 void f()
5 {
6 }
```

Однако если в одном модуле объявить переменную `x`, но не инициализировать ее, то компоновщик выберет имя, строго определенное в другом модуле (правило 2):

```
 1 /* foo3.c */
 2 #include <stdio.h>
 3 void f(void);
 4
 5 int x = 15213;
 6
 7 int main()
 8 {
 9     f();
10     printf("x = %d\n", x);
11     return 0;
12 }
```

```
1 /* bar3.c */
2 int x;
3
4 void f()
5 {
6     x = 15212;
7 }
```

Во время выполнения функция `f` заменит значение 15213 в переменной `x` на 15212, что может оказаться неприятной неожиданностью для автора функции `main`. Обратите внимание, что компоновщик обычно не выводит никаких сообщений, обнаружив несколько разных определений `x`:

```
linux> gcc -o foobar3 foo3.c bar3.c
linux> ./foobar3
x = 15212
```

То же может произойти при наличии двух слабых определений x (правило 3):

```
 1 /* foo4.c */
 2 #include <stdio.h>
 3 void f(void);
 4
 5 int x;
 6
 7 int main()
 8 {
 9     x = 15213;
10     f();
11     printf("x = %d\n", x);
12     return 0;
13 }
```

```
1 /* bar4.c */
2 int x;
3
4 void f()
5 {
6     x = 15212;
7 }
```

Применение правил 2 и 3 может привести к появлению некоторых коварных ошибок во время выполнения, малопонятных неопытному программисту, особенно если повторные определения объявляют имена с другими типами. Рассмотрим следующий пример, где в одном модуле переменная x определена с типом int, а в другом с типом double:

```
 1 /* foo5.c */
 2 #include <stdio.h>
 3 void f(void);
 4
 5 int y = 15212;
 6 int x = 15213;
 7
 8 int main()
 9 {
10     f();
11     printf("x = 0x%x y = 0x%x \n",
12            x, y);
13     return 0;
14 }
```

```
1 /* bar5.c */
2 double x;
3
4 void f()
5 {
6     x = -0.0;
7 }
```

На машине x86-64/Linux для переменной типа double отводится 8 байт, а для переменной типа int – 4 байта. В нашей системе переменная x хранится в памяти по адресу 0x601020, а переменная y – по адресу 0x601024. Таким образом, операция присваивания

`x = -0.0` в строке 6 в модуле `bar5.c` изменит содержимое ячеек памяти, занимаемых переменными `x` и `y` (строки 5 и 6 в модуле `foo5.c`)!

```
linux> gcc -Wall -Og -o foobar5 foo5.c bar5.c
/usr/bin/ld: Warning: alignment 4 of symbol 'x' in /tmp/cclUFK5g.o
is smaller than 8 in /tmp/ccbTLcb9.o
linux> ./foobar5
x = 0x0 y = 0x80000000
```

Это тонкая и опасная ошибка, особенно потому, что вызывает только вывод предупреждения компоновщика и проявляется позже – во время выполнения программы, далеко от того места, где возникла. В больших системах с сотнями модулей ошибки такого рода чрезвычайно трудно находить и устранять, особенно потому, что многие программисты не знают, как работают компоновщики, и игнорируют предупреждения компилятора. Если появляется сомнение, вызовите компоновщик с ключами, такими как `-fno-common`, чтобы потребовать генерировать ошибку при встрече многократных определений глобальных имен. Или используйте параметр `-Werror`, чтобы преобразовать любые предупреждения в ошибки.

В разделе 7.5 мы видели, как компилятор связывает имена с секциями `COMMON` и `.bss`, используя, казалось бы, произвольное соглашение. В действительности это соглашение обусловлено тем, что в некоторых случаях компоновщик позволяет определять одинаковые глобальные имена сразу в нескольких модулях. Когда компилятор транслирует какой-то модуль и встречает слабо определенное имя, скажем `x`, он не знает, определяют ли другие модули имя `x`, и потому не может предсказать, какой из нескольких экземпляров `x` выберет компоновщик. Поэтому компилятор перекладывает принятие решения на компоновщика, связывая `x` с секцией `COMMON`. С другой стороны, если переменная `x` инициализируется нулевым значением, то это строго определенное имя (и, согласно правилу 2, должно быть уникальным), поэтому компилятор может уверенно связать его с секцией `.bss`. Точно так же статические имена уникальны по своей природе, поэтому компилятор может уверенно связать их с секцией `.data` или `.bss`.

Упражнение 7.2 (решение в конце главы)

В этом упражнении запись `REF(x.i)` → `DEF(x.k)` означает, что компоновщик связывает произвольную ссылку на имя `x` в модуле `i` с определением `x` в модуле `k`. Для каждого последующего примера используйте эту нотацию, чтобы показать, как редактор связей разрешил бы ссылки на многократно определенное имя в каждом модуле. Если вы предполагаете, что на этапе компоновки возникнет ошибка (правило 1), то впишите слово «ERROR». Если вы считаете, что компоновщик произвольно выберет одно из определений (правило 3), то впишите слово «UNKNOWN».

1.

```
/* Модуль 1 */              /* Модуль 2 */
int main()                  int main;
{                           int p2()
}                           {
                            }
```

(a) `REF(main.1)` → `DEF(` _____`.`_____ `)`
(b) `REF(main.2)` → `DEF(` _____`.`_____ `)`

2.

```
/* Модуль 1 */              /* Модуль 2 */
int main()                  int main = 1;
{                           int p2()
}                           {
                            }
```

(a) REF(main.1) → DEF(_____._____)
(b) REF(main.2) → DEF(_____._____)

3.

```
/* Модуль 1 */          /* Модуль 2 */
int x;                  double x = 1.0;
int main()              int p2()
{                       {
}                       }
```

(a) REF(main.1) → DEF(_____._____)
(b) REF(main.2) → DEF(_____._____)

7.6.2. Связывание со статическими библиотеками

До сих пор мы предполагали, что компоновщик получает набор перемещаемых объектных файлов и связывает их вместе в выходной выполняемый файл. Практически все системы компиляции обеспечивают механизм, позволяющий упаковывать взаимосвязанные объектные модули в единый файл, называемый *статической библиотекой*, который можно затем передать компоновщику. Когда компоновщик связывает выходной выполняемый файл, он копирует из библиотеки только те объектные модули, на которые имеются ссылки из прикладной программы.

Почему системы поддерживают понятие библиотек? Рассмотрим стандарт ISO C99, определяющий широкий набор функций стандартного ввода/вывода, операций над строками и целочисленных математических функций, таких как `atoi`, `printf`, `scanf`, `strcpy` и `rand`. Они доступны каждой программе на языке C в библиотеке `libc.a`. ISO C99 определяет также обширный набор математических функций для операций с числами с плавающей точкой, таких как `sin`, `cos` и `sqrt`, в библиотеке `libm.a`.

Рассмотрим различные подходы, которые могли бы использовать разработчики компиляторов, чтобы дать пользователям доступ к этим функциям, не прибегая к использованию статических библиотек. Первый возможный подход: сконструировать компилятор, распознающий вызовы стандартных функций и добавляющий в программу соответствующий код. В языке Pascal, который предусматривает наличие небольшого набора стандартных функций, принят именно такой подход, но он неприемлем для языка C ввиду большого количества функций, определяемых стандартом языка. Выбор этого подхода существенно усложнил бы компилятор и потребовал изменять версию компилятора при добавлении новых, удалении старых или изменении существующих функций. Прикладным программистам, однако, такой подход был бы весьма удобен, потому что стандартные функции постоянно были бы доступны.

Другой подход: поместить все стандартные функции языка C в единственный перемещаемый объектный модуль, скажем `libc.o`, который прикладные программисты могли бы связывать со своими выполняемыми модулями:

```
linux> gcc main.c /usr/lib/libc.o
```

Преимущество этого подхода в том, что он отделяет реализацию стандартных функций от реализации компилятора и все еще достаточно удобен для программистов. Однако он имеет пару больших недостатков. Во-первых, каждый выполняемый файл будет содержать копии всех стандартных функций, что чрезвычайно расточительно с точки зрения использования дискового пространства. (В нашей системе `libc.a` занимает около 5 Мбайт и `libm.a` еще около 2 Мбайт.) Хуже того, каждая выполняющаяся программа тоже будет содержать копии всех стандартных функций в памяти, что было бы чрезвычайно расточительно, с точки зрения использования памяти. Во-вторых, любое изменение в любой стандартной функции, сколь бы незначительным оно ни было, потребует

от разработчиков библиотеки повторной компиляции всех файлов с исходным кодом, а это довольно продолжительная операция, которая усложнила бы разработку и сопровождение стандартных функций.

Мы могли бы ослабить эти проблемы, создав для каждой стандартной функции отдельный перемещаемый файл и сохранив эти файлы в известном каталоге. Однако этот подход потребовал бы от прикладных программистов явно компоновать свои программы с соответствующими объектными модулями, а это довольно трудоемкий процесс, при выполнении которого легко ошибиться:

```
linux> gcc main.c /usr/lib/printf.o /usr/lib/scanf.o ...
```

Для устранения неудобств и недостатков этих подходов была разработана концепция статической библиотеки. Взаимосвязанные функции можно собрать в отдельные объектные модули и затем упаковать в единственный файл статической библиотеки. Впоследствии прикладные программы смогут получить доступ к любым библиотечным функциям, для чего достаточно указать в командной строке единственное имя файла. Например, программу, использующую функции из стандартной и математической библиотек языка C, можно скомпилировать и скомпоновать с помощью такой команды:

```
linux> gcc main.c /usr/lib/libm.a /usr/lib/libc.a
```

Во время связывания компоновщик копирует только те объектные модули, на которые имеются ссылки в программе, что уменьшает размер выполняемого файла на диске и в памяти. С другой стороны, от прикладного программиста требуется включить имена лишь нескольких файлов библиотек. (Фактически драйверы компиляторов C всегда передают `libc.a` компоновщику, поэтому ссылка на `libc.a`, о которой говорилось выше, не нужна.)

В Linux статические библиотеки хранятся на диске в файлах, имеющих определенный формат, который называют *архивным*. Архив – это коллекция перемещаемых объектных файлов с заголовком, описывающим размер и адрес каждого элемента объектного файла. Имена архивных файлов обозначаются с использованием расширения `.a`. Чтобы сделать обсуждение библиотек более конкретным, предположим, что у нас есть пара процедур для обработки векторов, представленные в листинге 7.4. Каждая процедура определена в своем объектном модуле, выполняет операцию с парой векторов и сохраняет результат в выходном векторе. Дополнительно каждая процедура запоминает, сколько раз она была вызвана, увеличивая счетчик в глобальной переменной. (Это пригодится нам, когда мы перейдем обсуждению идеи позиционно-независимого кода в разделе 7.12.)

Листинг 7.4. Объектные файлы, составляющие библиотеку `libvector`

(a) addvec.o

code/link/addvec.c

```
1 int addcnt = 0;
2
3 void addvec(int *x, int *y,
4 int *z, int n)
5 {
6     int i;
7
8     addcnt++;
9
10    for (i = 0; i < n; i++)
11        z[i] = x[i] + y[i];
12 }
```

code/link/addvec.c

(b) multvec.o

code/link/multvec.c

```
1 int multcnt = 0;
2
3 void multvec(int *x, int *y,
4 int *z, int n)
5 {
6     int i;
7
8     multcnt++;
9
10    for (i = 0; i < n; i++)
11        z[i] = x[i] * y[i];
12 }
```

code/link/multvec.c

Создать статическую библиотеку из этих функций можно с помощью инструмента AR:

```
linux> gcc -c addvec.c multvec.c
linux> ar rcs libvector.a addvec.o multvec.o
```

Чтобы использовать эту библиотеку, можно написать приложение, как, например, `main2.c` в листинге 7.5, вызывающее библиотечную функцию `addvec` (заголовочный файл `vector.h` определяет прототипы функций из `libvector.a`).

Листинг 7.5. Пример программы 2. Эта программа вызывает функцию из библиотеки `libvector`

ccode/link/main2.c

```
 1 #include <stdio.h>
 2 #include "vector.h"
 3
 4 int x[2] = {1, 2};
 5 int y[2] = {3, 4};
 6 int z[2];
 7
 8 int main()
 9 {
10     addvec(x, y, z, 2);
11     printf("z = [%d %d]\n", z[0], z[1]);
12     return 0;
13 }
```

ccode/link/main2.c

Чтобы собрать выполняемый файл, нужно скомпилировать и связать входные файлы `main2.o` и `libvector.a`:

```
linux> gcc -c main2.c
linux> gcc -static -o prog2c main2.o ./libvector.a
```

или так:

```
linux> gcc -c main2.c
linux> gcc -static -o prog2c main2.o -L. -lvector
```

На рис. 7.3 приводится обобщенная схема сборки программы. Параметр `-static` сообщает драйверу компилятора, что компоновщик должен собрать полностью связанный выполняемый объектный файл, который можно загрузить в память и запустить на выполнение без всякой дополнительной обработки во время загрузки. Аргумент `-lvector` – это более краткая форма записи `libvector.a`, а параметр `-L.` сообщает компоновщику, что тот должен искать `libvector.a` в текущем каталоге.

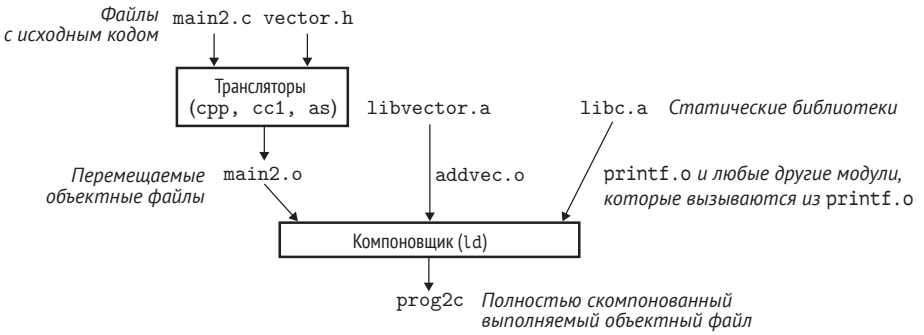


Рис. 7.3. Связывание со статическими библиотеками

Обрабатывая объектные файлы, компоновщик обнаруживает, что в `main2.o` имеется ссылка на имя `addvec`, определенное в `addvec.o`, поэтому он копирует `addvec.o` в выполняемый файл. Так как эта программа не ссылается на имена, определенные в `multvec.o`, компоновщик *не* копирует этот модуль в выполняемый файл. Кроме того, компоновщик копирует в выполняемый файл модуль `printf.o` из `libc.a` и еще несколько других модулей.

7.6.3. Как компоновщики разрешают ссылки на статические библиотеки

Статические библиотеки – полезные и необходимые инструменты, но они также являются источником недопонимания, если программист не имеет полного представления, как действует компоновщик Linux при разрешении внешних ссылок. При разрешении ссылок компоновщик просматривает перемещаемые объектные файлы и архивы слева направо, в той же последовательности, в какой они указаны в командной строке драйвера компилятора. (Драйвер автоматически преобразует любые файлы `.c`, указанные в командной строке, в файлы `.o`.) В процессе поиска имен компоновщик поддерживает набор *E* перемещаемых объектных файлов, которые будут объединены в выполняемый файл, набор *U* еще не найденных имен (на которые есть ссылки, но которые еще не определены) и набор имен *D*, которые были определены в предыдущих входных файлах. Первоначально *E*, *U* и *D* – пустые.

- Для каждого входного файла *f* в командной строке компоновщик определяет, является ли *f* объектным файлом или архивом. Если *f* – объектный файл, то компоновщик добавляет его в набор *E*, корректирует *U* и *D*, чтобы отразить определения имени и ссылки в *f*, и переходит к следующему входному файлу.
- Если *f* – это архив, то компоновщик пытается отыскать в нем имена из набора неразрешенных ссылок *U*. Если какой-то член архива, например *m*, определяет имя, присутствующее в *U*, то *m* будет добавлен в *E* и компоновщик скорректирует наборы *U* и *D* так, чтобы отразить найденные определения имен в *m* и разрешенные ссылки на них. Этот процесс повторяется для всех членов архива, пока не будет достигнута точка, когда *U* и *D* перестают изменяться. После этого все члены объектных файлов, отсутствующие в *E*, просто отбрасываются, и компоновщик переходит к следующему входному файлу.
- Если после обработки входных файлов в наборе *U* еще останутся неразрешенные ссылки, то компоновщик выведет сообщение об ошибке и завершится, иначе – объединит и переместит объектные файлы в *E*, чтобы собрать выходной выполняемый файл.

К сожалению, этот алгоритм может приводить к некоторым странным ошибкам времени компоновки из-за того, что порядок следования библиотек и объектных файлов в командной строке имеет существенное значение. Если библиотека, где определено некоторое имя, появляется в командной строке перед объектным файлом, который ссылается на это имя, то ссылка не будет разрешена и связывание завершится ошибкой. Например, взгляните на следующий пример:

```
linux> gcc -static ./libvector.a main2.c
/tmp/cc9XH6Rp.o: In function ‘main’:
/tmp/cc9XH6Rp.o(.text+0x18): undefined reference to ‘addvec’
```

Что здесь не так? После обработки `libvector.a` набор *U* останется пустым, поэтому в *E* не будет добавлено никаких объектных модулей из `libvector.a`. Как следствие ссылка на `addvec` не будет разрешена, и компоновщик выведет сообщение об ошибке и завершится.

Общее правило для библиотек – размещать их в конце командной строки. Если члены различных библиотек не зависят друг от друга, в том смысле, что если одна библиотека не содержит ссылок на имена, определенные в другой библиотеке, то эти библиотеки можно поместить в конец командной строки в любом порядке. Напротив, если библиотеки зависят друг от друга, то они должны быть указаны в командной строке в таком порядке, чтобы ссылки на имена в одном архиве предшествовали их определениям в другом. Например, предположим, что `foo.c` вызывает функции из `libx.a` и `libz.a`, а те вызывают функции из `liby.a`. В таком случае `libx.a` и `libz.a` должны предшествовать `liby.a` в командной строке:

```
linux> gcc foo.c libx.a libz.a liby.a
```

Библиотеки могут повторяться в командной строке, если это необходимо для удовлетворения циклических зависимостей. Например, если `foo.c` вызывает функцию из библиотеки `libx.a`, которая вызывает функцию из `liby.a`, а та вызывает функцию из `libx.a`, то `libx.a` необходимо повторно указать в командной строке:

```
linux> gcc foo.c libx.a liby.a libx.a
```

Как вариант можно объединить `libx.a` и `liby.a` в один архив.

Упражнение 7.3 (решение в конце главы)

Пусть `a` и `b` обозначают объектные модули или статические библиотеки в текущем каталоге, и пусть `a` → `b` обозначает зависимость `a` от `b`, то есть `b` определяет имя, на которое ссылается `a`. Для каждого из следующих сценариев укажите минимальную командную строку (с наименьшим количеством объектных файлов и библиотек), которая позволит статическому компоновщику разрешить все ссылки.

1. `p.o` → `libx.a`
2. `p.o` → `libx.a` → `liby.a`
3. `p.o` → `libx.a` → `liby.a` и `liby.a` → `libx.a` → `p.o`

7.7. Перемещение

По завершении этапа разрешения ссылок каждая ссылка в программном коде будет связана с точно одним определением имени (элементом в таблице имен в одном из объектных модулей). К этому моменту компоновщик будет знать точные размеры секций программного кода и данных и готов приступить к этапу перемещения, в ходе которого объединит входные модули и назначит адрес каждому имени. Перемещение выполняется в два шага.

1. *Перемещение секций и определений имен*. На этом шаге компоновщик объединяет все секции одного типа в новую составную секцию того же типа. Например, все секции `.data` из входных модулей объединяются в единую секцию `.data` в выходном выполняемом объектном файле. Затем компоновщик назначает адреса времени выполнения новым объединенным секциям, включающим секции, определяемые входными модулями, и каждому имени, определенному во входных модулях. По завершении этого шага каждая (машинная) инструкция и каждая глобальная переменная в программе получат свой уникальный адрес.
2. *Перемещение ссылок на имена внутри секций*. На этом шаге компоновщик изменяет все ссылки на имена в программном коде и в секции данных так, чтобы они указывали на корректные адреса времени выполнения. Для этого компо-

новщик использует структуры данных в перемещаемых объектных модулях, так называемые записи перемещения, которые описываются далее.

7.7.1. Записи перемещения

Когда ассемблер генерирует объектный модуль, он не знает, где код и данные этого модуля будут находиться в памяти во время выполнения. Он также не знает адресов внешних функций или глобальных переменных, на которые ссылается данный модуль. Поэтому всякий раз, когда, столкнувшись со ссылкой на объект, чье местоположение неизвестно, ассемблер генерирует запись перемещения, чтобы сообщить компоновщику, как следует изменить ссылку при объединении объектных файлов в выполняемый. Записи перемещения для программного кода помещаются в секцию `.rel.text`; записи перемещения для инициализированных данных – в `.rel.data`.

В листинге 7.6 показан формат записи перемещения ELF. Поле `offset` – это смещение ссылки внутри секции, которую нужно изменить. Поле `symbol` определяет имя, на которое должна указывать ссылка. Поле `type` сообщает компоновщику, как следует изменить новую ссылку. Поле `addend` – константа со знаком, которая используется при перемещениях некоторых видов для корректировки значения измененной ссылки.

Листинг 7.6. Запись перемещения в формате ELF. Каждая запись определяет ссылку, которую необходимо переместить, и как получить измененную ссылку

ccode/link/elfstructs.c

```
1 typedef struct {
2     long offset;  /* Смещение перемещаемой ссылки */
3     long type:32, /* Тип перемещения*/
4     symbol:32;    /* Индекс в таблице имен */
5     long addend;  /* Постоянная часть выражения перемещения */
6 } Elf64_Rela;
```

ccode/link/elfstructs.c

Формат ELF определяет 32 разных вида перемещений, порой весьма загадочных. Мы остановимся только на двух самых главных типах.

1. `R_X86_64_PC32`. Перемещает ссылку, использующую 32-битный адрес относительно счетчика инструкций (PC). Как рассказывалось в разделе 3.6.4, относительный адрес определяет смещение относительно текущего значения счетчика инструкций. Когда процессор выполняет инструкцию, использующую относительную адресацию, он формирует *эффективный адрес* (например, адрес процедуры в инструкции `call`), прибавляя 32-битное значение, закодированное в инструкции, к текущему значению счетчика инструкций, которое всегда является адресом следующей инструкции в памяти.
2. `R_X86_64_32`. Перемещает ссылку, использующую абсолютный 32-битный адрес. При использовании абсолютной адресации процессор использует 32-битное значение, закодированное в инструкции, непосредственно как эффективный адрес, без всяких модификаций.

Эти два типа перемещений поддерживают *малая модель кода* x86-64, в которой предполагается, что общий размер кода и данных в исполняемом объектном файле занимает меньше 2 Гбайт и к ним можно обращаться, используя 32-битные относительные адреса. Малая модель кода применяется в GCC по умолчанию. Программы с размером более 2 Гбайт можно скомпилировать с использованием флагов `-mcmodel=medium` (*средняя модель кода*) и `-mcmodel=large` (*большая модель кода*), но мы не будем их обсуждать.

7.7.2. Перемещение ссылок

В листинге 7.7 показан псевдокод, демонстрирующий алгоритм перемещения ссылок, используемый компоновщиком. Строки 1 и 2 организуют перебор каждой секции `s` и каждой записи перемещения `r` в секциях. Для конкретности предположим, что каждая секция `s` – это массив байтов, а каждая запись перемещения `r` – структура типа `Elf64_Rela` из листинга 7.6. Предположим также, что к тому времени, когда начнет выполняться алгоритм, компоновщик уже выбрал адреса времени выполнения для каждой секции (`ADDR(s)`) и каждого имени (`ADDR(r.symbol)`). В строке 3 вычисляется адрес в массиве `s` 4-байтных ссылок, которые должны быть перемещены. В случае относительной адресации перемещение будет выполнено в строках 5–9. В случае абсолютной адресации перемещение будет выполнено в строках 11–13.

Листинг 7.7. Алгоритм перемещения

```
 1 foreach section s {
 2   foreach relocation entry r {
 3       refptr = s + r.offset; /* указатель на перемещаемую ссылку */
 4
 5       /* Перемещение ссылок с относительной адресацией */
 6       if (r.type == R_X86_64_PC32) {
 7           refaddr = ADDR(s) + r.offset; /* ссылка на адрес времени выполнения */
 8           *refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr);
 9       }
10
11       /* Перемещение ссылок с абсолютной адресацией */
12       if (r.type == R_X86_64_32)
13           *refptr = (unsigned) (ADDR(r.symbol) + r.addend);
14   }
15 }
```

Давайте посмотрим, как компоновщик использует этот алгоритм для перемещения ссылок в нашем примере программы в листинге 7.1. В листинге 7.8 показан дизассемблированный код из `main.o`, сгенерированный инструментом GNU OBJDUMP (`objdump -dx main.o`).

Листинг 7.8. Записи перемещения из `main.o`. Исходный код на C приводится в листинге 7.1

ccode/link/main-relo.d

```
1  0000000000000000 <main>:
2     0:  48 83 ec 08            sub $0x8,%rsp
3     4:  be 02 00 00 00         mov $0x2,%esi
4     9:  bf 00 00 00 00         mov $0x0,%edi         %edi = &array
5                         a: R_X86_64_32 array         Запись перемещения
6     e:  e8 00 00 00 00         callq 13 <main+0x13>  sum()
7                         f: R_X86_64_PC32 sum-0x4     Запись перемещения
8    13:  48 83 c4 08            add $0x8,%rsp
9    17:  c3 retq
```

ccode/link/main-relo.d

Функция `main` ссылается на два глобальных имени, `array` и `sum`. Для каждой ссылки ассемблер сгенерировал запись перемещения, которая отображается в следующей строке[2]. Записи перемещения сообщают компоновщику, что ссылку на `sum` следует пере-

[2] Напомню, что записи перемещения и инструкции фактически хранятся в разных секциях объектного файла. Инструмент OBJDUMP отображает их вместе для удобства.

местить с использованием 32-битного относительного адреса, а ссылку на array – с использованием 32-битного абсолютного адреса. В следующих двух разделах подробно рассказывается, как компоновщик перемещает эти ссылки.

Перемещение относительных ссылок

В строке 6 в листинге 7.8 функция main вызывает функцию sum, которая определена в модуле sum.o. Инструкция call находится со смещением 0xe от начала раздела и состоит из 1-байтного кода операции 0xe8, за которым следуют 4 байта относительной ссылки на вызываемую функцию sum.

Соответствующая запись перемещения r состоит из четырех полей:

```
r.offset = 0xf
r.symbol = sum
r.type   = R_X86_64_PC32
r.addend = -4
```

Согласно этим полям, компоновщик должен изменить 32-битную относительную ссылку, начиная со смещения 0xf, чтобы во время выполнения она указывала на процедуру sum. Теперь предположим, что компоновщик определил, что

```
ADDR(s) = ADDR(.text) = 0x4004d0
```

и

```
ADDR(r.symbol) = ADDR(sum) = 0x4004e8
```

Применяя алгоритм из листинга 7.8, компоновщик сначала вычисляет адрес времени выполнения данной ссылки (строка 7):

```
refaddr = ADDR(s) + r.offset
        = 0x4004d0 + 0xf
        = 0x4004df
```

Затем скорректируем саму ссылку так, чтобы она указывала на подпрограмму sum во время выполнения (строка 8):

```
*refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr)
        = (unsigned) (0x4004e8       + (-4)     - 0x4004df)
        = (unsigned) (0x5)
```

В получившемся выполняемом объектном файле инструкция call приобретает следующий переместимый вид:

```
4004de:  e8 05 00 00 00     callq  4004e8 <sum>    sum()
```

Во время выполнения инструкция call будет находиться по адресу 0x4004de. Когда процессор приступит к ее выполнению, в счетчике инструкций PC будет храниться значение 0x4004e3 – адрес инструкции, следующей непосредственно за инструкцией call. Чтобы выполнить инструкцию вызова, процессор предпримет следующие действия:

1. Втолкнет PC в стек.
2. PC ← PC + 0x5 = 0x4004e3 + 0x5 = 0x4004e8.

То есть следующей будет выполнена первая инструкция в процедуре sum, что нам и нужно!

Перемещение абсолютных ссылок

Перемещение абсолютных ссылок несложно. Например, в строке 4 в листинге 7.8 инструкция mov копирует адрес array (32-битное непосредственное значение) в ре-

гистр %edi. Инструкция mov начинается со смещения 0x9 от начала секции и состоит из 1-байтного кода операции 0xbf, за которым следуют 4 байта абсолютной ссылки на array.

Соответствующая запись перемещения r состоит из четырех полей:

```
r.offset = 0xa
r.symbol = array
r.type   = R_X86_64_32
r.addend = 0
```

Согласно этим полям, компоновщик должен изменить абсолютную ссылку, начиная со смещения 0xa, чтобы во время выполнения она указывала на первый байт array. Теперь предположим, что компоновщик определил, что

```
ADDR(r.symbol) = ADDR(array) = 0x601018
```

Выполнив строку 13 в листинге 7.7, компоновщик скорректирует ссылку:

```
*refptr = (unsigned) (ADDR(r.symbol) + r.addend)
        = (unsigned) (0x601018       + 0)
        = (unsigned) (0x601018)
```

В получившемся выполняемом объектном файле ссылка приобретает следующий переместимый вид:

```
4004d9:  bf 18 10 60 00     mov    $0x601018,%edi    %edi = &array
```

В конечном итоге получатся секции .text и .data, как показано в листинге 7.9. Во время загрузки программы загрузчик скопирует байты из этих разделов непосредственно в память и запустит инструкции на выполнение без каких-либо дополнительных изменений.

Листинг 7.9. Секции .text и .data в выполняемом файле prog после перемещения. Исходный код на C показан в листинге 7.1

(a) Секция .text после перемещения

```
 1  00000000004004d0 <main>:
 2    4004d0:  48 83 ec 08       sub    $0x8,%rsp
 3    4004d4:  be 02 00 00 00    mov    $0x2,%esi
 4    4004d9:  bf 18 10 60 00    mov    $0x601018,%edi   %edi = &array
 5    4004de:  e8 05 00 00 00    callq  4004e8 <sum>     sum()
 6    4004e3:  48 83 c4 08       add    $0x8,%rsp
 7    4004e7:  c3 retq

 8  00000000004004e8 <sum>:
 9    4004e8:  b8 00 00 00 00    mov    $0x0,%eax
10    4004ed:  ba 00 00 00 00    mov    $0x0,%edx
11    4004f2:  eb 09             jmp    4004fd <sum+0x15>
12    4004f4:  48 63 ca          movslq %edx,%rcx
13    4004f7:  03 04 8f          add    (%rdi,%rcx,4),%eax
14    4004fa:  83 c2 01          add    $0x1,%edx
15    4004fd:  39 f2             cmp    %esi,%edx
16    4004ff:  7c f3             jl     4004f4 <sum+0xc>
17    400501:  f3 c3             repz retq
```

(b) Секция .data после перемещения

```
1  0000000000601018 <array>:
2    601018:  01 00 00 00 02 00 00 00
```

Упражнение 7.4 (решение в конце главы)

Исследуйте перемещенный код в листинге 7.9 (а) и ответьте на следующие вопросы:

1. Определите адрес (шестнадцатеричный) ссылки на `sum` в строке 5.
2. Определите значение (шестнадцатеричное) ссылки на `sum` в строке 5.

Упражнение 7.5 (решение в конце главы)

Пусть для вызова функции `swap` в объектном файле `m.o` (листинг 7.3)

```
9:  e8 00 00 00 00      callq e <main+0xe>    swap()
```

имеется следующая запись перемещения:

```
r.offset = 0xa
r.symbol = swap
r.type   = R_X86_64_PC32
r.addend = -4
```

Предположим, что компоновщик перемещает секцию `.text` из `m.o` в адрес `0x4004d0` и секцию `.text` из `swap.o` в адрес `0x4004e8`. Какое значение получит ссылка на `swap` в инструкции `callq` после перемещения?

7.8. Выполняемые объектные файлы

Мы уже видели, как компоновщик объединяет несколько объектных модулей в единый выполняемый объектный файл. Наша программа на языке C, которая начинала свое существование как простая совокупность текстовых файлов, была преобразована в единственный двоичный файл, содержащий всю информацию, необходимую для загрузки программы в память и ее выполнения. На рис. 7.4 приводится обобщенная картина организации типичного выполняемого файла ELF со всей заключенной в нем информацией.

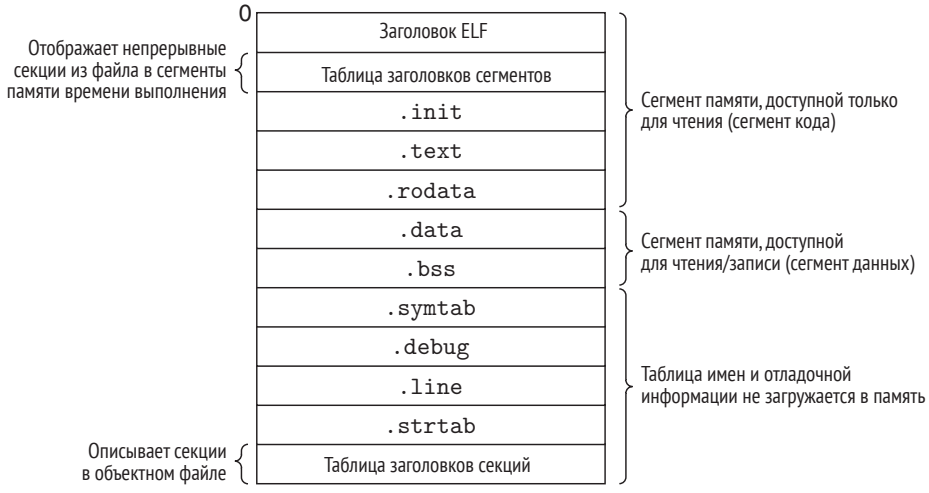


Рис. 7.4. Типичный выполняемый объектный файл ELF

Формат выполняемого объектного файла напоминает формат перемещаемого объектного файла. Заголовок ELF описывает общий формат файла. Он также определяет *точку входа* в программу – адрес инструкции, с которой начинается выполнение программы. Секции `.text`, `.rodata` и `.data` подобны аналогичным секциям перемещаемого объектного файла, за исключением того, что они уже перемещены в адреса памяти времени выполнения. Секция `.init` определяет небольшую функцию с именем `_init`, которая вызывается кодом инициализации программы. Поскольку выполняемый файл является *полностью связанным*, он не нуждается в секциях `.rel`.

Формат выполняемых файлов ELF разработан так, чтобы упростить их загрузку в память и смежные фрагменты выполняемого файла отображались в смежные сегменты памяти. Это отображение описывается в *таблице заголовков сегментов*. В листинге 7.10 показана часть таблицы заголовков сегментов из выполняемого файла нашего примера программы `prog`, как ее выводит утилита OBJDUMP.

Листинг 7.10. Таблица заголовков сегментов из примера выполняемого файла `prog`. Здесь `off` означает смещение (offset) от начала объектного файла; `vaddr/paddr` – адрес в памяти; `align` – требование к выравниванию; `filesz` – размер сегмента в объектном файле; `memsz` – размер сегмента в памяти; `flags` – разрешения времени выполнения

ccode/link/prog-exe.d

```
   Сегмент кода, доступен только для чтения
1  LOAD off  0x0000000000000000 vaddr 0x0000000000400000 paddr 0x0000000000400000 align 2**21
2       filesz 0x000000000000069c memsz 0x000000000000069c flags r-x

   Сегмент данных, доступен для чтения и записи
3  LOAD off  0x00000000000000df8 vaddr 0x0000000000600df8 paddr 0x0000000000600df8 align 2**21
4       filesz 0x0000000000000228 memsz 0x0000000000000230 flags rw
```

ccode/link/prog-exe.d

Из таблицы заголовков сегментов мы видим, что два сегмента памяти будут инициализированы содержимым выполняемого объектного файла. Строки 1 и 2 сообщают, что первый сегмент (*сегмент кода*) имеет разрешения на чтение и выполнение, начинается в памяти с адреса `0x400000`, имеет общий размер `0x69c` байт и будет инициализирован первыми `0x69c` байтами из выполняемого объектного файла, которые включают заголовок ELF, таблицу заголовков сегментов и секции `.init`, `.text` и `.rodata`.

Строки 3 и 4 сообщают, что второй сегмент (*сегмент данных*) имеет разрешения на чтение и запись, начинается в памяти с адреса `0x600df8`, занимает `0x230` байт и будет инициализирован `0x228` байтами из секции `.data` в объектном файле. Остальные 8 байт в этом сегменте соответствуют данным в секции `.bss` и будут инициализированы нулями во время выполнения.

Для любого сегмента *s* компоновщик должен выбрать такой начальный адрес `vaddr`, что

$$\texttt{vaddr} \bmod \texttt{align} = \texttt{off} \bmod \texttt{align}$$

где `off` – смещение первой секции в объектном файле, соответствующей сегменту, а `align` – выравнивание, указанное в заголовке программы (2^{21} = `0x200000`). Например, для сегмента данных в листинге 7.10:

$$\texttt{vaddr} \bmod \texttt{align} = \texttt{0x600df8} \bmod \texttt{0x200000} = \texttt{0xdf8}$$

и

$$\texttt{off} \bmod \texttt{align} = \texttt{0xdf8} \bmod \texttt{0x200000} = \texttt{0xdf8}$$

Это требование к выравниванию является оптимизацией, позволяющей эффективно загружать сегменты объектного файла в память во время выполнения программы. Причина такого поведения связана с организацией виртуальной памяти в виде больших непрерывных фрагментов с размерами, кратными степени двойки. Подробнее о виртуальной памяти рассказывается в главе 9.

7.9. Загрузка выполняемых объектных файлов

Чтобы запустить выполняемый объектный файл `prog`, достаточно ввести в командной строке его имя:

```
linux> ./prog
```

Поскольку имя `prog` не соответствует никакой встроенной команде, то командная оболочка предполагает, что `prog` – это выполняемый объектный файл, и передает его имя *загрузчику* операционной системы. Любая программа в Linux может запустить загрузчик, вызвав функцию `execve`, которую мы подробно обсудим в разделе 8.4.6. Загрузчик копирует код и данные из выполняемого объектного файла на диске в память и затем запускает программу, передав управление ее первой инструкции в *точке входа*. Этот процесс копирования программы в память и последующего выполнения называется *загрузкой*.

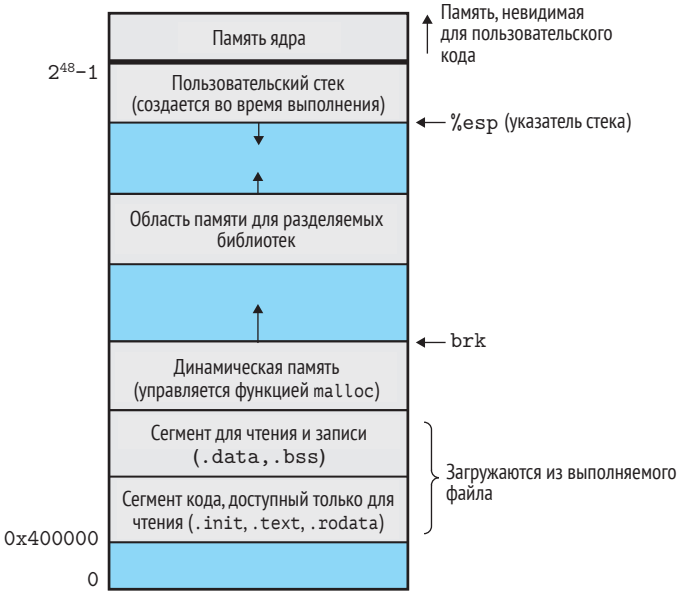


Рис. 7.5. Образ памяти времени выполнения в Linux с архитектурой x86-64. Разрывы, обусловленные требованиями к выравниванию и рандомизацией, здесь не показаны

Каждая программа в Linux имеет образ памяти времени выполнения, подобный изображенному на рис. 7.5. В системах Linux с архитектурой x86-64 сегмент программного кода всегда начинается с адреса `0x400000`. За ним следует сегмент данных. *Динамическая память* (куча) следует за сегментом данных и растет вверх по мере обращений к библиотечной функции `malloc` (подробнее функцию `malloc` и динамическую память мы обсудим в разделе 9.9). Далее следует область, зарезервированная для разделяемых модулей. Пользовательский стек начинается с наибольшего допустимого адреса ($2^{48} - 1$) и

растет вниз (в сторону меньших адресов памяти). Область выше стека, начиная с адреса 2^{48}, зарезервирована для кода и данных резидентной части операционной системы, известной как *ядро*.

Для простоты сегменты кучи, данных и кода изображены на рис. 7.5 как примыкающие друг к другу, а вершина стека размещена по наибольшему адресу, допустимому в пространстве пользователя. Но в действительности между сегментами кода и данных имеется разрыв, обусловленный требованиями к выравниванию сегмента `.data` (раздел 7.8). Кроме того, компоновщик использует рандомизацию адресного пространства (раздел 3.10.4), выбирая адреса времени выполнения для стека, разделяемых библиотек и кучи. Несмотря на то что расположение этих областей меняется при каждом запуске программы, их относительное положение одинаково.

После вызова загрузчик создает образ памяти, показанной на рис. 7.5. Руководствуясь таблицей заголовков сегментов в выполняемом файле, он копирует его сегменты программного кода и данных в память и затем передает управление в точку входа, которая всегда совпадает с адресом функции `_start`. Эта функция определяется в системном объектном файле `crt1.o` и одинакова для всех программ на языке C. Функция `_start` вызывает *системную функцию запуска* `__libc_start_main`, которая определена в `libc.so`. Он инициализирует среду выполнения, вызывает пользовательскую функцию `main`, обрабатывает возвращаемое ею значение и, при необходимости, возвращает управление ядру.

Как на самом деле работает загрузчик

Наше описание загрузки концептуально корректно, но в нем опущены многие детали. Чтобы понять, как в действительности происходит загрузка, необходимо знать и понимать такие концепции, как *процессы*, *виртуальная память* и *отображение памяти*, которые мы еще не обсуждали. Позже, когда мы познакомимся с этими понятиями в главах 8 и 9, мы вновь вернемся к вопросу загрузки и постепенно откроем эту тайну.

Для тех, кому не терпится, вот краткий обзор фактической работы загрузчика. Каждая программа в системе Linux выполняется в контексте процесса со своим собственным виртуальным адресным пространством. Когда командная оболочка запускает программу, родительский процесс запускает дочерний процесс – точную копию самого себя. Дочерний запускает загрузчик обращением к системному вызову `execve`. Загрузчик удаляет сегменты виртуальной памяти дочернего процесса и создает новый набор сегментов для программного кода, данных, кучи и стека. Новые сегменты стека и кучи инициализируются нулевыми байтами. Новые сегменты кода и данных инициализируются значением содержимого выполняемого файла путем отображения страниц виртуального адресного пространства во фрагменты выполняемого файла. Затем загрузчик передает управление в точку `_start`, что в конечном итоге приводит к вызову процедуры `main` приложения. Кроме некоторой информации из заголовка, никакие другие данные не копируются с диска в память. Копирование откладывается до того момента, когда процессор обратится к отображенной виртуальной странице, и в этот момент операционная система автоматически загрузит страницу с диска в память, используя свой механизм подкачки страниц.

7.10. Динамическое связывание с разделяемыми библиотеками

Статические библиотеки, которые мы изучали в разделе 7.6.2, решают многие проблемы, связанные с созданием больших коллекций функций для прикладных программ. Однако статические библиотеки все же имеют некоторые существенные недостат-

ки – они, как и любое другое программное обеспечение, должны поддерживаться и обновляться. Если прикладной программист захочет использовать самую последнюю версию библиотеки, он должен иметь возможность узнать, какие библиотеки изменились, и затем заново скомпоновать свои программы, использующие измененные библиотеки.

Еще одна проблема в том, что почти каждая программа на C использует стандартные функции ввода/вывода, такие как `printf` и `scanf`. Во время выполнения программный код этих функций будет дублироваться в сегменте `.text` каждого выполняющегося процесса. В типичной системе выполняются сотни процессов, поэтому дублирование кода стандартных функций может приводить к существенным затратам такого дефицитного ресурса, как память. (Интересное свойство памяти – она *всегда* будет оставаться дефицитным ресурсом, сколько бы ее ни было в системе. Это свойство также присуще дисковому пространству и мусорным ведрам на кухнях.)

Разделяемые библиотеки (совместно используемые библиотеки) – одна из недавних новинок, созданная с целью избавиться от недостатков статических библиотек. Разделяемая библиотека – это объектный модуль, который во время выполнения может быть загружен в произвольный адрес памяти и связан с выполняющейся программой. Этот процесс известен как *динамическое связывание* и выполняется программой, называемой *динамическим компоновщиком*. Разделяемые библиотеки также называют *разделяемыми объектами* (shared objects) и в системах Linux обычно обозначают расширением файла `.so`. В операционных системах Microsoft разделяемые библиотеки тоже широко используются и называются библиотеками DLL (Dynamic Link Libraries – динамически связываемые библиотеки).

Разделяемые библиотеки могут «разделяться» (совместно использоваться) двумя способами. Во-первых, в любой файловой системе имеется точно один файл `.so` определенной библиотеки. Программный код и данные в этом файле совместно используются всеми выполняемыми объектными файлами, которые ссылаются на данную библиотеку, в отличие от содержимого статических библиотек, которое копируется и встраивается в выполняемые файлы, ссылающиеся на них. Во-вторых, единственная копия секции `.text` разделяемой библиотеки в памяти может совместно использоваться разными выполняющимися процессами. Мы рассмотрим этот вопрос более подробно после знакомства с виртуальной памятью в главе 9.

На рис. 7.6 приводится обобщенная схема процесса динамического связывания для примера программы из листинга 7.5. Чтобы построить разделяемую библиотеку `libvector.so` функций для выполнения операций с векторами из листинга 7.4, мы должны передать драйверу компилятора специальную директиву для компоновщика:

```
linux> gcc -shared -fpic -o libvector.so addvec.c multvec.c
```

Флаг `-fpic` предписывает компилятору сгенерировать *позиционно-независимый код* (Position-Independent Code, PIC). Флаг `-shared` предписывает компоновщику создать разделяемый объектный файл. После создания библиотеки ее можно связать с нашей программой из листинга на рис. 7.5:

```
linux> gcc -o prog2l main2.c ./libvector.so
```

Эта команда создаст выполняемый объектный файл `prog2l` в форме, позволяющей скомпоновать его с `libvector.so` во время выполнения. Основная идея – настроить одну часть связей статически, в процессе создания выполняемого файла, а другую – после загрузки программы. Важно отметить, что в этот момент никакие секции – кода или данных – из `libvector.so` не копируются в выполняемый файл `prog2l`. Компоновщик копирует лишь некоторую информацию из таблицы перемещений и таблицы имен, которая позволит разрешить ссылки на код и данные в `libvector.so` во время загрузки.

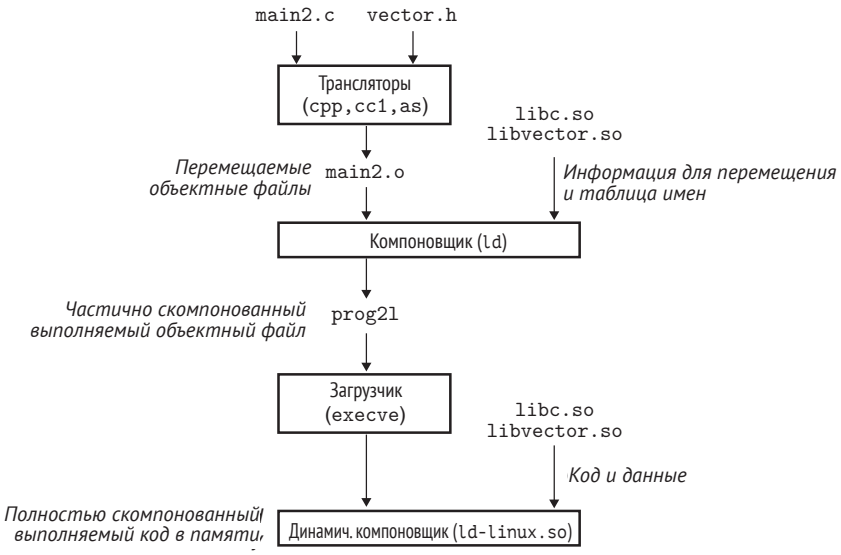


Рис. 7.6. Динамическое связывание с разделяемыми библиотеками

Когда загрузчик загружает и затем запускает выполняемый модуль `prog2l`, он загружает частично скомпонованный выполняемый файл `prog2l`, используя методы, описанные в разделе 7.9. Затем замечает, что `prog2l` содержит секцию `.interp`, содержащую путь к динамическому компоновщику, который сам является разделяемым объектом (например, `ld-linux.so` в системах Linux). Вместо того чтобы передать управление приложению, как это обычно делается, загрузчик загружает и запускает динамический компоновщик, который затем завершает задачу компоновки, выполняя следующие действия:

- перемещение кода и данных из `libc.so` в некоторый сегмент памяти;
- перемещение кода и данных из `libvector.so` в другой сегмент памяти;
- перемещение каждой ссылки в `prog2l` на имена, определенные в `libc.so` и `libvector.so`.

Наконец, динамический компоновщик передает управление приложению. С этого момента местоположение разделяемых библиотек фиксируется и не изменяется в течение всего времени выполнения программы.

7.11. Загрузка и связывание с разделяемыми библиотеками из приложений

Вплоть до этого момента мы рассматривали сценарий, когда динамический загрузчик загружает и связывает разделяемые библиотеки в процессе загрузки приложения, непосредственно перед его выполнением. Однако приложение может также обратиться к динамическому загрузчику, чтобы загрузить и задействовать произвольную разделяемую библиотеку, уже после запуска.

Динамическое связывание – мощный и полезный инструмент. Вот только некоторые примеры из практики:

- *распространение программного обеспечения*. Разработчики приложений для Microsoft Windows часто используют разделяемые библиотеки для распростра-

нения обновлений своих программ. Они генерируют новую копию разделяемой библиотеки, после чего пользователи могут загрузить и установить ее взамен текущей версии. После этого при следующем запуске приложение автоматически загрузит и будет использовать новую версию разделяемой библиотеки;

- *построение высокопроизводительных веб-серверов*. Многие веб-серверы управляют *динамическим контентом*, таким как персонифицированные веб-страницы, учетные записи и рекламные объявления. Ранние веб-серверы генерировали динамический контент с помощью функций `fork` и `execve`, используя дочерний процесс для выполнения «программы CGI». Но современные высокопроизводительные веб-серверы используют намного более эффективный и совершенный подход, основанный на динамическом связывании.

  Идея состоит в том, чтобы упаковывать каждую функцию, генерирующую динамический контент, в разделяемую библиотеку. Получив запрос от веб-браузера, сервер динамически загрузит соответствующую функцию и вызовет ее напрямую, без запуска дочернего процесса с помощью `fork` и `execve`. Функция кешируется в адресном пространстве сервера, поэтому последующие аналогичные запросы будут обрабатываться намного быстрее, простым вызовом этой функции. Это может оказать существенное влияние на эффективность работы сайта. Кроме того, можно модифицировать существующие функции и добавлять новые прямо во время выполнения, не останавливая сервер.

Системы класса Linux предлагают простой интерфейс к динамическому компоновщику, который позволяет прикладным программам загружать и связывать разделяемые библиотеки во время выполнения.

```
#include <dlfcn.h>

void *dlopen(const char *filename, int flag);

                    Возвращает указатель на дескриптор в случае успеха
                    и NULL в случае ошибки
```

Функция `dlopen` загружает и связывает разделяемую библиотеку `filename`. Внешние имена в `filename` разрешаются с использованием библиотек, ранее открытых с флагом `RTLD_GLOBAL`. Если текущий выполняемый модуль скомпилирован с флагом `-rdynamic`, то его глобальные имена также доступны для разрешения. Аргумент `flag` должен включать флаг `RTLD_NOW`, сообщающий компоновщику, что тот должен разрешать ссылки на внешние имена непосредственно, или `RTLD_LAZY`, сообщающий, что компоновщик должен отложить разрешение ссылок до момента обращения к данной библиотеке. Любой из этих флагов можно объединить логической операцией ИЛИ вместе с флагом `RTLD_GLOBAL`.

```
#include <dlfcn.h>

void *dlsym(void *handle, char *symbol);

                    Возвращает указатель на symbol в случае успеха
                    и NULL в случае ошибки
```

Функция `disym` принимает аргумент `handle` с дескриптором предварительно открытой разделяемой библиотеки и идентификатор `symbol` и возвращает адрес, соответствующий этому идентификатору, если тот существует, или `NULL` в противном случае.

```
#include <dlfcn.h>

int dlclose (void *handle);
```

Возвращает 0 в случае успеха, –1 в случае ошибки

Функция `dlclose` выгружает разделяемую библиотеку, если она нигде больше не используется.

```
#include <dlfcn.h>

const char *dlerror(void);
```

Возвращает сообщение об ошибке, если предыдущий вызов `dlopen`, `dlsym` или `dlclose` потерпел неудачу, и `NULL`, если предыдущие вызовы перечисленных функций завершились успехом

Функция `dlerror` возвращает строку, описывающую самую последнюю ошибку, возникшую при вызове `dlopen`, `dlsym` или `dlclose`, или `NULL`, если ошибки не было.

В листинге 7.11 показано, как можно использовать этот интерфейс для динамического связывания нашей разделяемой библиотеки `libvector.so` с приложением во время выполнения и затем вызвать процедуру `addvec`. Чтобы скомпилировать программу, GCC следует вызвать, как показано ниже:

```
linux> gcc -rdynamic -o prog2r dll.c -ldl
```

Листинг 7.11. Пример программы 3. Динамическая загрузка и связывание разделяемой библиотеки `libvector.so` во время выполнения

ccode/link/dll.c

```
 1 #include <stdio.h>
 2 #include <stdlib.h>
 3 #include <dlfcn.h>
 4
 5 int x[2] = {1, 2};
 6 int y[2] = {3, 4};
 7 int z[2];
 8
 9 int main()
10 {
11     void *handle;
12     void (*addvec)(int *, int *, int *, int);
13     char *error;
14
15     /* Динамически загрузить разделяемую библиотеку с функцией addvec() */
16     handle = dlopen("./libvector.so", RTLD_LAZY);
17     if (!handle) {
18         fprintf(stderr, "%s\n", dlerror());
19         exit(1);
20     }
21
22     /* Получить указатель на функцию addvec() */
23     addvec = dlsym(handle, "addvec");
24     if ((error = dlerror()) != NULL) {
25         fprintf(stderr, "%s\n", error);
```

```
26          exit(1);
27      }
28
29      /* Теперь можно вызвать addvec(), как самую обычную функцию */
30      addvec(x, y, z, 2);
31      printf("z = [%d %d]\n", z[0], z[1]);
32
33      /* Выгрузить разделяемую библиотеку */
34      if (dlclose(handle) < 0) {
35          fprintf(stderr, "%s\n", dlerror());
36          exit(1);
37      }
38      return 0;
39  }
```

ccode/link/dll.c

Разделяемые библиотеки и интерфейс языка Java

Язык Java определяет стандарт соглашения вызовов двоичных функций *Java Native Interface* (*JNI*). Этот стандарт допускает вызов функций, написанных на языках C и C++ из программ на Java. Основная идея JNI состоит в том, чтобы скомпилировать функцию на языке C (например, `foo`) в разделяемую библиотеку (например, `foo.so`). Когда программа на Java попытается вызвать функцию `foo`, интерпретатор Java вызовет интерфейс `dlopen` (или подобный ему), чтобы динамически загрузить и связать `foo.so`, а затем вызвать `foo`.

7.12. Перемещаемый программный код

Основное назначение разделяемых библиотек – позволить нескольким процессам совместно использовать один и тот же библиотечный код и таким образом сэкономить дорогостоящий ресурс памяти. Но как несколько процессов могут вместе использовать единственную копию программного кода? Одно из возможных решений – назначить каждой разделяемой библиотеке определенный участок адресного пространства, а затем потребовать, чтобы загрузчик всегда загружал разделяемую библиотеку по этому адресу. Такое решение кажется довольно простым, но в действительности порождает серьезные проблемы. Подобное использование адресного пространства очень неэффективно, потому что некоторые его области будут зарезервированы для возможного использования, даже если процесс не использует библиотеку. Кроме того, таким механизмом трудно управлять. Мы не можем гарантировать, что никакие области не будут перекрываться. После каждого изменения библиотеки придется удостовериться, что она все еще умещается в пределы назначенной ей области памяти, иначе придется подыскать для нее новый участок. А если мы создадим новую библиотеку, то придется найти место и для нее. Через какое-то время мы получим сотни библиотек и их версий и будет очень трудно предотвратить фрагментацию адресного пространства на множество маленьких и непригодных для использования «дырок». Хуже того, в разных системах библиотеки придется закреплять за разными областями памяти, что вызовет еще большую головную боль.

Чтобы избежать подобных проблем, современные системы компилируют библиотечный код так, чтобы его можно было загрузить по любому адресу без модификации с использованием компоновщика. При таком подходе мы получаем единственную копию сегмента разделяемого кода, который может использоваться неограниченным количеством процессов. (При этом, конечно же, каждый процесс получит свою копию сегмента данных, доступных для чтения/записи.)

Программный код, который можно загружать без перемещения, называется *позиционно-независимым* (Position-Independent Code, PIC). В системе компиляции GNU такой код генерируется при компиляции с параметром `-fpic`. Разделяемые библиотеки всегда должны компилироваться с этим параметром.

В системах x86-64 ссылки на идентификаторы в том же выполняемом объектном модуле не требуют специальной подготовки, чтобы получить позиционно-независимый код. При компиляции этих ссылок уже используется относительная адресация и перемещение выполняется статическим компоновщиком при сборке объектного файла. Но ссылки на внешние процедуры и глобальные переменные, определяемые в разделяемой библиотеке, требуют дополнительной обработки с применением некоторых специальных приемов, о которых рассказывается далее.

Позиционно-независимые ссылки на данные

Компиляторы генерируют позиционно-независимые ссылки на глобальные переменные, используя следующий интересный факт: независимо от того, куда в памяти загружается объектный модуль (в том числе и разделяемые), сегмент данных всегда будет следовать сразу за сегментом кода. Соответственно, *расстояние* от любой инструкции в сегменте кода до любой переменной в сегменте данных во время выполнения остается постоянным и не зависит от абсолютных адресов начала сегментов кода и данных.

Чтобы получить позиционно-независимые ссылки на глобальные переменные, компилятор создает в начале сегмента данных *глобальную таблицу смещений* (Global Offset Table, GOT). Таблица GOT содержит запись для каждого глобального объекта данных (процедуру или глобальную переменную), на который имеется ссылка в объектном модуле. Компилятор также генерирует запись перемещения для каждого элемента таблицы GOT. Во время загрузки динамический компоновщик преобразует каждую запись в GOT так, чтобы она содержала абсолютный адрес соответствующего объекта. Каждый объектный модуль, ссылающийся на глобальные данные, имеет свою таблицу GOT.

На рис. 7.7 показана таблица GOT для нашего примера разделяемой библиотеки `libvector.so`. Подпрограмма `addvec` получает адрес глобальной переменной `addcnt` из `GOT[3]`, а затем увеличивает значение `addcnt` в памяти. Ключевая идея заключается в том, что относительное смещение ссылки на `GOT[3]` является постоянным во время выполнения.

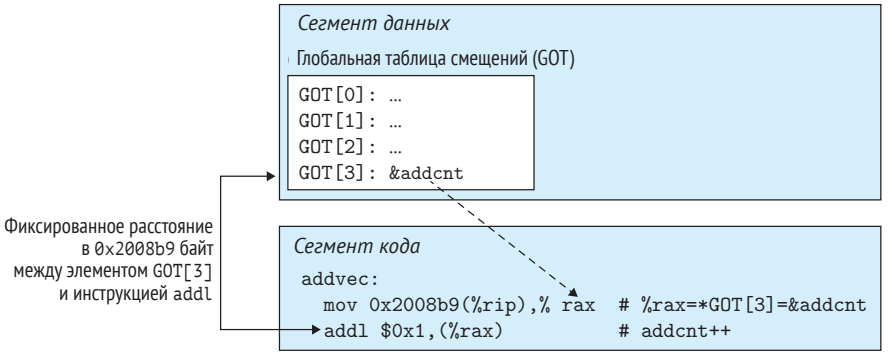


Рис. 7.7. Использование таблицы GOT для ссылки на глобальную переменную. Подпрограмма `addvec` в `libvector.so` применяет косвенную ссылку на `addcnt` через таблицу GOT библиотеки `libvector.so`

Поскольку `addcnt` определяется модулем `libvector.so`, компилятор мог бы использовать постоянное расстояние между сегментами кода и данных, создав прямую относительную ссылку на `addcnt` и добавив для компоновщика задание на перемещение

при сборке разделяемого модуля. Однако если бы addcnt была определена другим разделяемым модулем, то потребовался бы косвенный доступ через GOT. В этом случае компилятор решил использовать наиболее универсальное решение – адресацию через GOT – для всех ссылок.

Позиционно-независимые ссылки на функции

Предположим, что программа вызывает функцию, которая определена в разделяемой библиотеке. Компилятор не может предсказать, по какому адресу окажется функция во время выполнения, потому что разделяемый модуль может быть загружен в любое место. Решить эту проблему можно созданием записи перемещения для ссылки, которую динамический компоновщик мог бы разрешить при загрузке программы. Однако такой подход не может считаться позиционно-независимым, так как требует от компоновщика изменить сегмент кода вызывающего модуля. Системы компиляции GNU решают эту проблему с помощью любопытного метода, получившего название *ленивое связывание*, в соответствии с которым связывание каждой процедуры откладывается до *первого* ее вызова.

Причина откладывания связывания заключается в том, что типичная прикладная программа вызывает лишь несколько из сотен или тысяч функций, экспортируемых такими универсальными библиотеками, как libc.so. Откладывая разрешение адреса функции до ее фактического вызова, динамический компоновщик может избежать сотен или тысяч ненужных перемещений во время загрузки. При первом вызове функции, конечно, возникают накладные расходы, но для всех последующих вызовов потребуется только одна инструкция и одна ссылка на память для извлечения адреса.

Отложенное связывание реализуется с использованием компактного, но немного сложного взаимодействия между двумя структурами данных: глобальной таблицей смещений GOT и *таблицей связывания процедур* (Procedure Linkage Table, PLT). Если объектный модуль вызывает какие-либо функции, находящиеся в разделяемых библиотеках, то он имеет свои таблицы GOT и PLT. GOT является частью сегмента данных, а PLT – частью сегмента кода.

На рис. 7.8 показано, как используются таблицы PLT и GOT для разрешения адреса функции во время выполнения. Для начала рассмотрим содержимое каждой из этих таблиц.

Таблица связывания процедур (PLT)

PLT – это массив 16-байтных записей. PLT[0] – специальная запись, вызывающая динамический компоновщик. Для каждой функции в разделяемой библиотеке, которая вызывается выполняемым файлом, имеется своя запись в таблице PLT. Каждая запись отвечает за вызов определенной функции. PLT[1] (здесь не показана) вызывает системную функцию запуска (__libc_start_main), которая инициализирует окружение выполнения, в свою очередь вызывает функцию main и обрабатывает возвращаемое ею значение. Записи, начинающиеся с PLT[2], соответствуют библиотечным функциям, вызываемым пользовательским кодом. В нашем примере PLT[2] вызывает addvec, а PLT[3] (здесь не показана) вызывает printf.

Таблица глобальных смещений (GOT)

Как мы уже видели, таблица GOT – это массив 8-байтных записей с адресами. При использовании в паре с PLT записи GOT[0] и GOT[1] содержат информацию, которая помогает динамическому компоновщику разрешать адреса функций. GOT[2] – это точка входа в динамический компоновщик в модуле ld-linux.so. Все остальные записи соответствуют вызываемым функциям, адреса которых необходимо разрешить во время выполнения. Каждой из них соответствует своя запись в PLT.

Например, GOT[4] и PLT[2] соответствуют функции addvec. Первоначально каждая запись в GOT указывает на вторую инструкцию в соответствующей записи PLT.

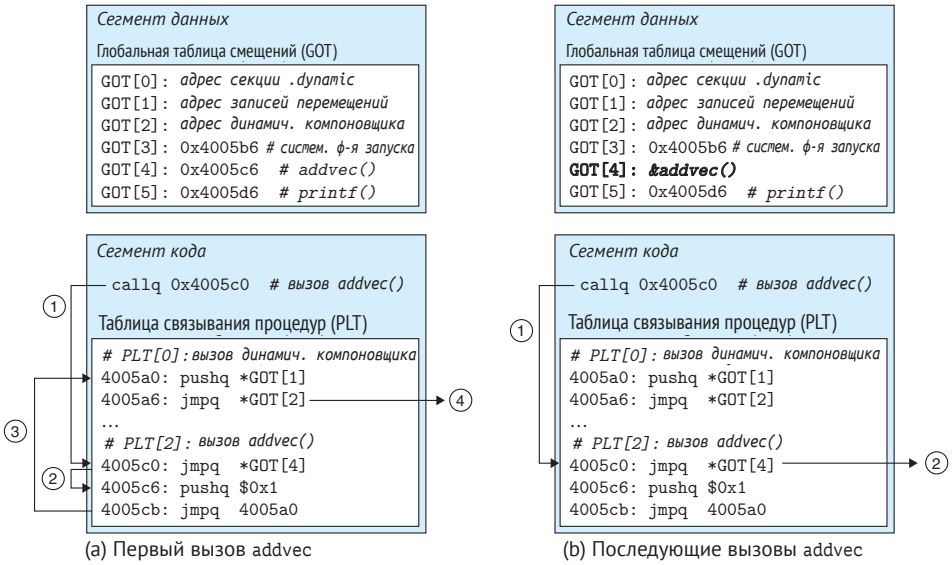


Рис. 7.8. Использование таблиц PLT и GOT для вызова внешних функций. Динамический компоновщик разрешает адрес addvec при первом вызове

На рис. 7.8 (a) показано, как механизм отложенного разрешения использует таблицы GOT и PLT для определения адреса функции addvec при первом вызове:

Шаг 1. Вместо прямого вызова addvec программа вызывает запись PLT[2], которая соответствует функции addvec.

Шаг 2. Первая инструкция в записи PLT выполняет косвенный переход через GOT[4]. Поскольку каждая запись в GOT первоначально ссылается на вторую инструкцию в соответствующей записи PLT, косвенный переход просто передает управление следующей инструкции в PLT[2].

Шаг 3. После помещения идентификационного номера addvec (0x1) в стек PLT[2] переходит к PLT[0].

Шаг 4. PLT[0] передает аргумент динамическому компоновщику косвенно через GOT[1], а затем косвенно переходит к динамическому компоновщику через GOT[2]. Динамический компоновщик извлекает две записи из стека для определения местоположения addvec во время выполнения, записывает полученный адрес в GOT[4] и передает управление addvec.

На рис. 7.8 (b) показано, как происходят все последующие вызовы addvec:

Шаг 1. Управление все так же передается записи PLT[2].

Шаг 2. Однако на этот раз косвенный переход через GOT[4] передает управление непосредственно addvec.

7.13. Подмена библиотечных функций

Компоновщики Linux поддерживают мощную методику *подмены библиотечных функций* (library interpositioning), которая позволяет перехватывать вызовы библиотечных

функций и вместо них выполнять свой код. Используя методику подмены, можно, например, подсчитать, сколько раз вызывалась конкретная библиотечная функция, проверить ее аргументы и результат и даже заменить ее совершенно другой реализацией.

Вот основная идея: для некоторой *целевой функции*, вызов которой нужно перехватить, создается функция-обертка с прототипом, идентичным прототипу целевой функции. Затем, используя определенный механизм подмены, вы обманываете систему, заставляя ее вызывать функцию-обертку вместо целевой функции. Функция-обертка обычно выполняет свою логику, а затем вызывает целевую функцию и передает возвращаемое ею значение обратно вызывающему коду.

Подмена может происходить во время компиляции, компоновки или выполнения, когда программа загружается и выполняется. Для изучения этих трех механизмов мы используем пример программы в листинге 7.12 (a). Она вызывает функции `malloc` и `free` из стандартной библиотеки C (`libc.so`). Вызов `malloc` выделяет из кучи блок размером 32 байта и возвращает указатель на этот блок. Вызов `free` возвращает блок обратно в кучу для использования последующими вызовами `malloc`. Наша цель – использовать механизм подмены для отслеживания вызовов `malloc` и `free` во время работы программы.

7.13.1. Подмена во время компиляции

В листинге 7.12 показано, как реализовать подмену на этапе компиляции с помощью препроцессора C. Каждая функция-обертка в `mymalloc.c` (листинг 7.12 (c)) вызывает целевую функцию, печатает трассировку и возвращает результат. Локальный заголовочный файл `malloc.h` (рис. 7.12 (b)) сообщает препроцессору, что тот должен заменить все вызовы целевой функции вызовом ее обертки. Вот как скомпилировать и скомпоновать программу:

```
linux> gcc -DCOMPILETIME -c mymalloc.c
linux> gcc -I. -o intc int.c mymalloc.o
```

Подмена обеспечивается параметром `-I.`, аргумент (точка) которого требует от препроцессора C искать заголовочные файлы (в том числе и `malloc.h`) сначала в текущем каталоге, а затем в обычных системных каталогах. Обратите внимание, что функции-обертки в `mymalloc.c` компилируются со стандартным заголовочным файлом `malloc.h`.

Эта программа выведет следующее:

```
linux> ./intc
malloc(32)=0x9ee010
free(0x9ee010)
```

Листинг 7.12. Подмена во время компиляции с помощью препроцессора C

(a) Пример программы `int.c`

code/link/interpose/int.c

```
1 #include <stdio.h>
2 #include <malloc.h>
3
4 int main()
5 {
6     int *p = malloc(32);
7     free(p);
8     return(0);
9 }
```

code/link/interpose/int.c

(b) Локальный файл `malloc.h`

code/link/interpose/malloc.h

```
1 #define malloc(size) mymalloc(size)
2 #define free(ptr) myfree(ptr)
3
4 void *mymalloc(size_t size);
5 void myfree(void *ptr);
```

code/link/interpose/malloc.h

(c) Функции-обертки в `mymalloc.c`

code/link/interpose/mymalloc.c

```
 1 #ifdef COMPILETIME
 2 #include <stdio.h>
 3 #include <malloc.h>
 4
 5 /* Функция-обертка для malloc */
 6 void *mymalloc(size_t size)
 7 {
 8     void *ptr = malloc(size);
 9     printf(“malloc(%d)=%p\n”,
10            (int)size, ptr);
11     return ptr;
12 }
13
14 /* Функция-обертка для free */
15 void myfree(void *ptr)
16 {
17     free(ptr);
18     printf("free(%p)\n", ptr);
19 }
20 #endif
```

code/link/interpose/mymalloc.c

7.13.2. Подмена во время компоновки

Статический компоновщик Linux поддерживает подмену во время компоновки, предлагая для этого флаг `--wrap f`. Этот флаг требует от компоновщика разрешать ссылки на имя `f` как на `__wrap_f` (два символа подчеркивания в начале) и ссылки на имя `__real_f` (два символа подчеркивания в начале) как на `f`. В листинге 7.13 показаны обертки для нашего примера программы.

Вот как скомпилировать исходные файлы в перемещаемые объектные файлы:

```
linux> gcc -DLINKTIME -c mymalloc.c
linux> gcc -c int.c
```

Листинг 7.13. Подмена во время компоновки с использованием флага `--wrap`

code/link/interpose/mymalloc.c

```
1 #ifdef LINKTIME
2 #include <stdio.h>
3
4 void *__real_malloc(size_t size);
5 void __real_free(void *ptr);
6
7 /* Функция-обертка для malloc */
```

```
 8 void *__wrap_malloc(size_t size)
 9 {
10     void *ptr = __real_malloc(size); /* Вызов malloc из libc */
11     printf("malloc(%d) = %p\n", (int)size, ptr);
12     return ptr;
13 }
14
15 /* Функция-обертка для free */
16 void __wrap_free(void *ptr)
17 {
18     __real_free(ptr); /* Вызов free из libc */
19     printf("free(%p)\n", ptr);
20 }
21 #endif
```

code/link/interpose/mymalloc.c

А вот как скомпоновать объектные файлы в выполняемый модуль:

```
linux> gcc -Wl,--wrap,malloc -Wl,--wrap,free -o intl int.o mymalloc.o
```

Параметр `-Wl, option` передает свой аргумент `option` компоновщику. Запятые в `option` заменяются пробелами. То есть параметр `-Wl,--wrap,malloc` передаст `--wrap malloc` компоновщику, аналогично сработает параметр `-Wl,--wrap,free`.

Эта программа выведет следующее:

```
linux> ./intl
malloc(32) = 0x18cf010
free(0x18cf010)
```

7.13.3. Подмена во время выполнения

Чтобы выполнить подмену во время компиляции, необходимо иметь доступ к файлам с исходным кодом программы. Для подмены во время компоновки необходимо иметь доступ к перемещаемым объектным файлам. Однако существует механизм подмены во время выполнения, которому достаточно иметь доступ только к выполняемому объектному файлу. Этот удивительный механизм основан на переменной окружения `LD_PRELOAD` динамического компоновщика.

Если в переменной окружения `LD_PRELOAD` задать список путей к разделяемым библиотекам (через пробелы или двоеточие), то при загрузке и выполнении программы динамический компоновщик (LD-LINUX.SO) сначала будет искать библиотеки в каталогах, перечисленных в `LD_PRELOAD`, и только потом в других стандартных каталогах. С помощью этого механизма можно перехватить вызов любой функции в любой библиотеке, включая `libc.so`, во время загрузки и в ходе выполнения любого выполняемого файла.

В листинге 7.14 показаны функции-обертки для `malloc` и `free`. Вызов `dlsym` в каждой из них возвращает указатель на целевую функцию в `libc`. Затем обертка вызывает целевую функцию, выводит свою информацию и возвращает значение, полученное от целевой функции.

Листинг 7.14. Подмена во время выполнения с помощью `LD_PRELOAD`

code/link/interpose/mymalloc.c

```
1 #ifdef RUNTIME
2 #define _GNU_SOURCE
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <dlfcn.h>
```

```
 6
 7 /* Функция-обертка для malloc */
 8 void *malloc(size_t size)
 9 {
10     void *(*mallocp)(size_t size);
11     char *error;
12
13     mallocp = dlsym(RTLD_NEXT, "malloc"); /* Получить адрес malloc в libc */
14     if ((error = dlerror()) != NULL) {
15         fputs(error, stderr);
16         exit(1);
17     }
18     char *ptr = mallocp(size); /* Вызов malloc из libc */
19     printf("malloc(%d) = %p\n", (int)size, ptr);
20     return ptr;
21 }
22
23 /* Функция-обертка free */
24 void free(void *ptr)
25 {
26     void (*freep)(void *) = NULL;
27     char *error;
28
29     if (!ptr)
30         return;
31
32     freep = dlsym(RTLD_NEXT, "free"); /* Получить адрес free в libc */
33     if ((error = dlerror()) != NULL) {
34         fputs(error, stderr);
35         exit(1);
36     }
37     freep(ptr); /* Вызвать free из libc */
38     printf("free(%p)\n", ptr);
39 }
40 #endif
```

code/link/interpose/mymalloc.c

Вот как собрать разделяемую библиотеку, содержащую функции-обертки:

```
linux> gcc -DRUNTIME -shared -fpic -o mymalloc.so mymalloc.c -ldl
```

Следующая команда скомпилирует основную программу:

```
linux> gcc -o intr int.c
```

Вот как запустить программу в командной оболочке bash[3]:

```
linux> LD_PRELOAD="./mymalloc.so" ./intr
malloc(32) = 0x1bf7010
free(0x1bf7010)
```

А так – в csh или в tcsh:

```
linux> (setenv LD_PRELOAD "./mymalloc.so"; ./intr; unsetenv LD_PRELOAD)
malloc(32) = 0x2157010
free(0x2157010)
```

Обратите внимание, что с помощью LD_PRELOAD можно подменить вызовы библиотечных функций из любых программ!

[3] Если вы не знаете, какая командная оболочка используется у вас, введите команду printenv SHELL.

```
linux> LD_PRELOAD="./mymalloc.so" /usr/bin/uptime
malloc(568) = 0x21bb010
free(0x21bb010)
malloc(15) = 0x21bb010
malloc(568) = 0x21bb030
malloc(2255) = 0x21bb270
free(0x21bb030)
malloc(20) = 0x21bb030
malloc(20) = 0x21bb050
malloc(20) = 0x21bb070
malloc(20) = 0x21bb090
malloc(20) = 0x21bb0b0
malloc(384) = 0x21bb0d0
 20:47:36 up 85 days,  6:04,  1 user,  load average: 0.10, 0.04, 0.05
```

7.14. Инструменты управления объектными файлами

В системе Linux имеется множество утилит, которые помогут вам исследовать объектные файлы и управлять ими. В частности, особенно полезным будет пакет GNU *binutils*, имеющийся в любой системе Linux.

- `AR` – создает статические библиотеки, вставляет, удаляет, перечисляет и извлекает элементы.
- `STRINGS` – выводит все печатаемые строки, имеющиеся в объектном файле.
- `STRIP` – удаляет таблицу имен из объектного файла.
- `NM` – перечисляет имена, определенные в таблице имен объектного файла.
- `SIZE` – перечисляет имена и размеры секций в объектном файле.
- `READELF` – показывает полную структуру объектного файла, включая информацию в заголовке ELF. Обладает возможностями утилит `SIZE` и `NM`.
- `OBJDUMP` – основа всех инструментов для работы с двоичными файлами. Может отобразить всю информацию об объектном файле. Чаще всего используется для дизассемблирования двоичного кода в секции `.text`.

Системы Linux также предоставляют программу LDD для управления разделяемыми библиотеками.

- `LDD` – перечисляет разделяемые библиотеки, необходимые выполняемому файлу во время выполнения.

7.15. Итоги

Компоновку можно выполнить во время компиляции с помощью статического компоновщика и во время загрузки с помощью динамического компоновщика. Компоновщики работают с двоичными файлами, которые называют объектными. Объектные файлы бывают трех видов: перемещаемые, выполняемые и разделяемые. Перемещаемые объектные файлы собираются с помощью статических компоновщиков в выполняемый объектный файл, который может быть загружен в память и запущен на выполнение. Разделяемые объектные файлы (разделяемые библиотеки) связываются и загружаются с помощью динамических загрузчиков во время выполнения – либо неявно, когда вызывающая программа загружается и начинает выполняться, либо явно, когда программа вызывает функции из библиотеки `dlopen`.

Компоновщики решают две основные задачи – разрешают ссылки, связывая каждое глобальное имя в объектном файле с уникальным определением, и выполняют переме-

щение, определяя для каждого имени окончательный адрес в памяти и соответственно модифицируя ссылки на них.

Статические компоновщики вызываются с помощью диспетчера компилятора, такого как GCC. Они связывают несколько перемещаемых объектных файлов в один выполняемый объектный файл. В различных объектных файлах может определяться одно и то же глобальное имя, и правила, используемые компоновщиками для разрешения ссылок на различные определения, могут привести к коварным ошибкам в пользовательских программах.

Несколько объектных файлов могут быть связаны в одну статическую библиотеку. Библиотеки используются редакторами связей для разрешения ссылок на имена в других объектных модулях. Последовательный просмотр слева направо, который используют многие редакторы связей для разрешения ссылок на имя, является еще одним источником труднообъяснимых ошибок.

Загрузчики отображают содержимое выполняемых файлов в память и запускают их. Компоновщики могут также создавать частично скомпонованные выполняемые объектные файлы с неразрешенными ссылками на процедуры и данные, определяемые в разделяемой библиотеке. Во время загрузки загрузчик отображает частично скомпонованный выполняемый файл в память и затем передает управление динамическому компоновщику, который завершает задачу связывания, загружая разделяемую библиотеку и перемещая ссылки в программе.

Разделяемые библиотеки, скомпилированные в позиционно-независимый код, могут загружаться в любое место адресного пространства и совместно использоваться несколькими процессами. Кроме того, приложения могут использовать динамический компоновщик во время выполнения, чтобы загрузить и связать функции и данные в разделяемых библиотеках.

Библиографические заметки

Вопросы компоновки плохо освещены в компьютерной литературе. Компоновка охватывает такие области знаний, как компиляция, компьютерная архитектура и операционные системы, поэтому для ее понимания требуется знать особенности генерирования объектного кода, программирования на машинном языке, реализации программ и виртуальной памяти. Компоновка не вписывается ни в одну из обычных специальностей в области компьютерных систем, и потому типичные вопросы, возникающие в этой сфере, недостаточно хорошо освещены в литературе. Впрочем, монография Левина (Levine) [69] предоставляет неплохое общее введение в предмет. Исходные спецификации ELF и DWARF для архитектуры IA32 (описывающие содержимое секций `.debug` и `.line`) описаны в [54]. Расширения формата ELF для архитектуры x86-64 описаны в [36]. Описание прикладного двоичного интерфейса (Application Binary Interface, ABI) для архитектуры x86-64 с соглашениями по компиляции, компоновке и выполнению программ для архитектуры x86-64 и правилами перемещения и оформления позиционно-независимого кода можно найти в [77].

Домашние задания

Упражнение 7.6 ♦

Исследуйте модуль `m.o` в листинге 7.3 и следующую версию функции `swap`, которая запоминает, сколько раз она вызывалась:

```
1 extern int buf[];
2
3 int *bufp0 = &buf[0];
4 static int *bufp1;
```

```
 5
 6 static void incr()
 7 {
 8     static int count=0;
 9
10     count++;
11 }
12
13 void swap()
14 {
15     int temp;
16
17     incr();
18     bufp1 = &buf[1];
19     temp = *bufp0;
20     *bufp0 = *bufp1;
21     *bufp1 = temp;
22 }
```

Для каждого имени, которое определено и на которое есть ссылка в `swap.o`, укажите, будет ли создана соответствующая запись в таблице имен в секции `.symtab` в модуле `swap.o`. Если да, то укажите модуль, в котором определено это имя (`swap.o` или `m.o`), тип имени (локальное, глобальное или внешнее), а также секцию (`.text`, `.data` или `.bss`), в которой это имя находится.

Имя	Имеется запись в .symtab в модуле swap.o?	Тип имени	Модуль, где определено	Секция
buf	_____	_____	_____	_____
bufp0	_____	_____	_____	_____
bufp1	_____	_____	_____	_____
swap	_____	_____	_____	_____
temp	_____	_____	_____	_____

Упражнение 7.7 ♦

Не изменяя никаких имен переменных, модифицируйте `bar5.c` в разделе 7.6.1 так, чтобы `foo5.c` выводила корректные значения `x` и `y` (т. е. шестнадцатеричные представления целых чисел 15213 и 15212).

Упражнение 7.8 ♦

Пусть `REF(x.i)` → `DEF(x.k)` обозначает тот факт, что компоновщик связывает некоторую ссылку на имя `x` в модуле `i` с определением `x` в модуле `k`. Используйте эту нотацию для каждого примера, чтобы указать, как компоновщик разрешит ссылки на неоднократно определенное имя в каждом модуле. Если на этапе компоновки возникнет ошибка (правило 1), впишите «ERROR». Если компоновщик произвольно выберет одно из возможных определений (правило 3), впишите «UNKNOWN».

1.

```
/* Модуль 1 */
int main()
{
}
```

```
/* Модуль 2 */
static int main=1;
int p2()
{
}
```

(a) REF(main.1) → DEF(_____.____)
(b) REF(main.2) → DEF(_____.____)

2.

```
/* Модуль 1 */          /* Модуль 2 */
int x;                  double x;
int main()              int p2()
{                       {
}                       }
```

(a) REF(x.1) → DEF(_____.____)
(b) REF(x.2) → DEF(_____.____)

3.

```
/* Модуль 1 */          /* Модуль 2 */
int x=1;                double x=1.0;
int main()              int p2()
{                       {
}                       }
```

(a) REF(x.1) → DEF(_____.____)
(b) REF(x.2) → DEF(_____.____)

Упражнение 7.9 ♦

Взгляните на следующую программу, состоящую из двух объектных модулей:

```
1 /* foo6.c */
2 void p2(void);
3
4 int main()
5 {
6     p2();
7     return 0;
8 }
```

```
1 /* bar6.c */
2 #include <stdio.h>
3
4 char main;
5
6 void p2()
7 {
8     printf("0x%x\n", main);
9 }
```

Если эта программа будет скомпилирована и запущена в системе Linux с архитектурой x86-64, она выведет строку `"0x48\n"` и нормально завершит работу, даже притом что функция `p2` нигде не инициализирует переменную `main`. Сможете ли вы объяснить такое поведение?

Упражнение 7.10 ♦♦

Пусть `a` и `b` обозначают объектные модули или статические библиотеки в текущем каталоге, и пусть `a` → `b` обозначает зависимость `a` от `b`, в том смысле, что `b` определяет имя, на которое имеется ссылка в `a`. Для каждого из следующих сценариев приведите минимальную команду (с наименьшим количеством объектных файлов и библиотек), которая позволит статическому компоновщику разрешить все ссылки на имена:

1. p.o → libx.a → p.o
2. p.o → libx.a → liby.a и liby.a → libx.a
3. p.o → libx.a → liby.a → libz.a и liby.a → libx.a → libz.a

Упражнение 7.11 ♦♦

Из заголовка сегмента в листинге 7.10 видно, что этот сегмент данных занимает в памяти 0x230 байт. Однако только первые 0x228 байт принадлежат секциям выполняемого файла. Чем вызвано это несоответствие?

Упражнение 7.12 ♦♦

Вызов функции swap в объектном файле m.o выглядит так (упражнение 7.6):

```
9:  e8 00 00 00 00      callq e <main+0xe>   swap()
```

Для этого вызова имеется следующая запись перемещения:

```
r.offset = 0xa
r.symbol = swap
r.type   = R_X86_64_PC32
r.addend = -4
```

1. Если предположить, что компоновщик перемещает .text в модуле m.o по адресу 0x4004e0 и swap – по адресу 0x4004f8, какое значение получит ссылка на swap в инструкции callq?
2. Если предположить, что компоновщик перемещает .text в модуле m.o по адресу 0x4004d0 и swap – по адресу 0x400500, какое значение получит ссылка на swap в инструкции callq?

Упражнение 7.13 ♦♦

Это упражнение поможет вам приобрести дополнительный опыт использования различных средств управления объектными файлами.

1. Сколько объектных файлов содержится в библиотеках libc.a и libm.a в вашей системе?
2. Будет ли отличаться машинный код, скомпилированный командой gcc -Og, от машинного кода, скомпилированного командой gcc -Og -g?
3. Какие разделяемые библиотеки использует драйвер GCC в вашей системе?

Решения упражнений

Решение упражнения 7.1

Цель этого упражнения – помочь вам понять связь между именами внутри компоновщика и идентификаторами переменных и функций в программном коде на C. Обратите внимание, что для локальной переменной temp в исходном коде на C компилятор не создает запись в таблице имен.

Имя	Имеется запись в .symtab?	Тип имени	Модуль, где определено	Секция
buf	Да	внешнее	m.o	.data
bufp0	Да	глобальное	swap.o	.data
bufp1	Да	глобальное	swap.o	COMMON
swap	Да	глобальное	swap.o	.text
temp	Да	–	–	–

Решение упражнения 7.2

Цель этого упражнения – помочь вам проверить свое понимание правил, которыми пользуется компоновщик в Unix при разрешении ссылок на глобальные имена, определяемые в более чем одном модуле. Знание и понимание этих правил поможет вам избежать некоторых неприятных ошибок при программировании.

1. Компоновщик выберет строго определенное имя из модуля 1, оставив в стороне слабо определенное имя в модуле 2 (правило 2):
   (a) `REF(main.1)` → `DEF(main.1)`
   (b) `REF(main.2)` → `DEF(main.1)`
2. Здесь следует вписать слово «ERROR», потому что оба модуля содержат строгие определения одного и того же имени `main` (правило 1).
3. Компоновщик выберет строго определенное имя из модуля 2, оставив в стороне слабо определенное имя в модуле 1 (правило 2):
   (a) `REF(x.1)` → `DEF(x.2)`
   (b) `REF(x.2)` → `DEF(x.2)`

Решение упражнения 7.3

Перечисление статических библиотек в командной строке в неправильном порядке является распространенным источником ошибок времени компоновки, которые ставят в тупик многих программистов. Однако если разобраться с тем, как компоновщик использует статические библиотеки для разрешения ссылок, все станет на свои места. Это небольшое упражнение поможет вам проверить ваше понимание данной идеи:

```
1. linux> gcc p.o libx.a
2. linux> gcc p.o libx.a liby.a
3. linux> gcc p.o libx.a liby.a libx.a
```

Решение упражнения 7.4

В этом упражнении предлагалось проанализировать листинг 7.9 (a). Его цель – дать вам некоторую практику чтения листингов дизассемблера и помочь сложить представление об относительной адресации.

1. Ссылка в строке 5 окажется по адресу `0x4004df`.
2. Ссылка в строке 5 получит шестнадцатеричное значение `0x5`. Напомню, что в дизассемблированном листинге значения ссылок выводятся с использованием обратного (little-endian) порядка следования байтов.

Решение упражнения 7.5

Это упражнение поможет вам проверить свое понимание, как компоновщик производит перемещение ссылок при использовании относительной адресации. Вам было дано:

```
ADDR(s) = ADDR(.text) = 0x4004d0
```

и

```
ADDR(r.symbol) = ADDR(swap) = 0x4004e8
```

Согласно алгоритму в листинге 7.7, компоновщик сначала вычислит адрес ссылки:

```
refaddr = ADDR(s)  + r.offset
        = 0x4004d0 + 0xa
        = 0x4004da
```

а затем обновит ссылку:

```
*refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr)
        = (unsigned) (0x4004e8       + (-4)     - 0x4004da)
        = (unsigned) (0xa)
```

В результате относительная ссылка на swap получит значение `0xa`:

```
4004d9:  e8 0a 00 00 00     callq 4004e8 <swap>
```